
RUST FOR BACKEND DEVELOPMENT

A Practical, Project-Based Course

From zero Rust knowledge to a deployed, authenticated, database-backed REST API — built one real concept at a time.

What you'll build: TaskFlow — a multi-user task management API using Axum, PostgreSQL (SQLx), and JWT authentication.

Format: 32 chapters + 5 appendices · concept explanations, annotated code, hands-on exercises in every chapter.

Prerequisites: comfort with any programming language (frontend/JS background is a plus) — no prior Rust or backend experience required.

Prepared for Siddhant · Rust Backend Development Course

How to Use This Course

This course is organized into six parts, moving from core language fundamentals to a complete, deployed backend service, followed by a set of deeper, optional topics for continued growth.

- **Part I — Rust Foundations (Ch. 1-8):** syntax, ownership, and the type system. Do every exercise here without skipping — ownership underpins everything that follows.
- **Part II — Intermediate Rust (Ch. 9-12):** error handling, traits/generics, project structure, and threads.
- **Part III — Building Backends (Ch. 13-16):** async/Tokio, the Axum web framework, PostgreSQL with SQLx, and JWT authentication — the core skills for shipping an API.
- **Part IV — Production & Capstone (Ch. 17-21):** centralized error handling, testing, the full TaskFlow build, Docker deployment, and a roadmap for continued practice.
- **Part V — Going Further (Ch. 22-26):** caching, security hardening, background jobs, WebSockets, and performance — optional but valuable once TaskFlow is working end to end.
- **Part VI — Architecture & Craft (Ch. 27-32):** design patterns, stack alternatives, CI/CD, reading compiler errors, and structuring larger codebases.

Every chapter follows the same rhythm: a plain-language explanation of the concept, one or more annotated, runnable code examples, and a short hands-on exercise. Type the code out yourself rather than copy-pasting — for a language this focused on the compiler catching your mistakes, making (and fixing) the mistakes is most of the learning.

TIP

You do not need to read Parts V and VI before building your first real project. Finish Part IV, ship TaskFlow, then return to whichever Part V/VI chapter matches whatever your next project actually needs.

Table of Contents

PART I — RUST FOUNDATIONS

- Chapter 1 Why Rust for Backend Development
- Chapter 2 Installing Rust and Setting Up Your Workspace
- Chapter 3 Variables, Data Types & Functions
- Chapter 4 Control Flow
- Chapter 5 Ownership, Borrowing & Lifetimes
- Chapter 6 Structs & Enums
- Chapter 7 Pattern Matching in Depth
- Chapter 8 Collections: Vec, HashMap, String

PART II — INTERMEDIATE RUST

- Chapter 9 Error Handling
- Chapter 10 Traits & Generics
- Chapter 11 Modules, Crates & Cargo Workspaces
- Chapter 12 Concurrency with Threads

PART III — BUILDING BACKENDS

- Chapter 13 Async/Await & Tokio
- Chapter 14 Building a Web Backend with Axum
- Chapter 15 Databases with SQLx & PostgreSQL
- Chapter 16 Authentication with JWT

PART IV — PRODUCTION & CAPSTONE

- Chapter 17 Middleware, Logging & Centralized Error Handling
- Chapter 18 Testing in Rust
- Chapter 19 Capstone Project: Building TaskFlow End-to-End
- Chapter 20 Deployment with Docker
- Chapter 21 Best Practices & Where to Go Next

PART V — GOING FURTHER

- Chapter 22 Caching with Redis
- Chapter 23 Rate Limiting & Security
- Chapter 24 Background Jobs & Scheduled Tasks
- Chapter 25 Real-Time Features with WebSockets
- Chapter 26 Performance, Profiling & Benchmarking

PART VI — ARCHITECTURE & CRAFT

Chapter 27 Common Design Patterns in Rust Backends
Chapter 28 Choosing Your Stack — Framework & Ecosystem Comparison
Chapter 29 CI/CD with GitHub Actions
Chapter 30 A Beginner's Field Guide to Compiler Errors
Chapter 31 gRPC & GraphQL in Rust
Chapter 32 Structuring Larger Rust Backends
Appendix A — Cheat Sheet & Further Resources
Appendix B — Rust Backend Interview Questions & Answers
Appendix C — Glossary of Terms
Appendix D — 20 Practice Projects to Build After This Course
Appendix E — HTTP Status Code Quick Reference

Why Rust for Backend Development

Rust is a systems programming language that gives you the speed of C/C++ with the safety guarantees normally found in languages like Java or Python — without needing a garbage collector. For backend development this combination is extremely attractive: your API server can handle huge traffic on small, cheap servers, and an entire category of bugs (null pointer crashes, data races, use-after-free) simply cannot compile.

What makes Rust different

- **No garbage collector, no manual `free()` either.** Rust tracks memory ownership at compile time, so it frees memory automatically and deterministically, with zero runtime overhead.
- **The compiler is your reviewer.** Rust's compiler (`rustc`) refuses to compile code with data races, dangling pointers, or many classes of logic bugs. This feels strict at first, but it means fewer 3am production incidents.
- **Fearless concurrency.** Because the compiler enforces ownership rules, you can write multi-threaded backend code without the usual fear of race conditions.
- **Performance close to C.** Rust compiles to native machine code with no interpreter and no VM. Companies like Discord, Cloudflare, and Amazon (AWS) run huge chunks of backend infrastructure in Rust specifically for this reason.
- **One toolchain for everything.** Cargo (Rust's package manager and build tool) handles dependencies, building, testing, formatting, and publishing — there's no separate npm/pip/Maven-style fragmentation.

Why a frontend/QA developer should learn Rust for backend

You already think in terms of components, state, and testing — which transfers directly. Rust backend code is organized similarly to well-structured JavaScript/TypeScript: small modules, clear function signatures, and strong typing (TypeScript users usually feel at home very quickly). The big shift is learning **ownership** — a concept unique to Rust — which this course spends real time on because everything else builds on top of it.

TIP

This course is built around one continuous real project — a **Task Management REST API called TaskFlow** — that grows chapter by chapter. By Chapter 19 you will have a complete, working, database-backed, authenticated backend service.

What you will be able to build after this course

- A production-style REST API using the **Axum** web framework
- Persistent storage with **PostgreSQL** via **SQLx**
- Token-based authentication using **JWT**

- Proper error handling, logging, and middleware
- Automated tests for your backend logic
- A Docker image you can deploy anywhere

Installing Rust and Setting Up Your Workspace

2.1 Installing Rust with rustup

Rust is installed and managed through **rustup**, a version manager similar in spirit to **nvm** for Node.js. It installs the compiler (**rustc**), the package manager (**cargo**), and lets you switch between stable, beta, and nightly toolchains.

Terminal — installing the Rust toolchain

```
# Linux / macOS
curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Windows: download and run rustup-init.exe from rustup.rs

# Verify installation
rustc --version
cargo --version
```

2.2 Cargo: your build tool and package manager

Cargo is the single command you will use for almost everything: creating projects, adding dependencies (called **crates**), compiling, running, and testing.

Command	What it does
cargo new my_app	Creates a new binary project
cargo new my_lib --lib	Creates a new library crate
cargo build	Compiles the project (debug mode)
cargo build --release	Compiles with full optimizations
cargo run	Compiles and runs the project
cargo check	Type-checks without producing a binary (fast)
cargo add	Adds a dependency to Cargo.toml
cargo test	Runs all tests
cargo fmt	Auto-formats your code
cargo clippy	Lints your code for common mistakes

2.3 Anatomy of a Cargo project

Project layout

```
my_app/
├── Cargo.toml # project metadata + dependencies (like package.json)
├── Cargo.lock # exact locked dependency versions (auto-generated)
├── src/
│   └── main.rs # entry point — fn main() lives here
```

Cargo.toml — example

```
[package]
name = "my_app"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { version = "1", features = ["derive"] }
```

2.4 Editor setup

- **VS Code + rust-analyzer** extension — the standard, gives autocomplete, inline errors, and go-to-definition.
- Enable **format on save** using *cargo fmt* so your code style stays consistent automatically.
- Install **clippy** (bundled with rustup) — it catches non-idiomatic patterns before they become habits.

2.5 Your first program

src/main.rs

```
fn main() {
    println!("Hello, backend world!");
}
```

Run it with:

Terminal

```
cargo run
```

■ Try it yourself

Install Rust with rustup and confirm `cargo --version` works.

Create a new project called `taskflow` with `cargo new taskflow` — this will be our project for the entire course.

Change the greeting message and run it again with `cargo run`.

Variables, Data Types & Functions

3.1 Variables are immutable by default

This is the first surprise for most newcomers: in Rust, variables cannot be changed after being set, unless you explicitly say so with **mut**. This default is intentional — it prevents an entire class of accidental-mutation bugs common in JavaScript.

Immutability by default

```
fn main() {
    let user_count = 5;
    // user_count = 6; // compile error: cannot assign twice to immutable variable

    let mut retry_count = 0;
    retry_count = 1; // works because of `mut`
}
```

3.2 Basic data types

Category	Types	Example
Integers	i8 i16 i32 i64 i128, u8 u16 u32 u64 u128, isize/usize	let age: u8 = 24;
Floats	f32, f64	let price: f64 = 19.99;
Boolean	bool	let is_active: bool = true;
Character	char (a single unicode scalar)	let grade: char = 'A';
String slice	&str; (borrowed, fixed text)	let name: &str; = "Sid";
Owned string	String (growable, heap-allocated)	let mut s = String::from("hi");

NOTE

Rust has two string types because of ownership. **&str;** is a read-only view into text (fast, borrowed). **String** owns its data on the heap and can grow — think of it like the difference between a read-only prop and local component state in React.

3.3 Functions

Functions and return expressions

```
fn add(a: i32, b: i32) -> i32 {
    a + b // no semicolon = this is the return value (an "expression")
}

fn greet(name: &str) {
```

```

    println!("Hello, {name}!");
}

fn main() {
    let sum = add(4, 5);
    greet("Siddhant");
    println!("Sum = {sum}");
}

```

Notice there is no **return** keyword needed on the last line of a function — dropping the semicolon turns a statement into the return value. This "everything is an expression" idea shows up constantly in Rust.

3.4 Constants and shadowing

const vs shadowing

```

const MAX_CONNECTIONS: u32 = 100; // must have an explicit type, known at compile time

fn main() {
    let x = 5;
    let x = x + 1; // "shadowing": a new variable, same name
    let x = x * 2;
    println!("{x}"); // prints 12
}

```

3.5 Tuples and arrays

Tuples and arrays

```

fn main() {
    // Tuple: fixed size, mixed types
    let user: (&str, u8, bool) = ("sid", 24, true);
    let (name, age, _active) = user; // destructuring

    // Array: fixed size, same type
    let scores: [i32; 3] = [90, 85, 77];
    println!("{name} is {age}, first score {}", scores[0]);
}

```

■ Practice

Write a function `celsius_to_fahrenheit(c: f64) -> f64` and call it with a few values.

Create a tuple representing a backend `Task``: (id: u32, title: &str, done: bool), then destructure it and print each field.

Control Flow

4.1 if / else — as an expression

if / else

```
fn main() {
    let age = 20;

    if age >= 18 {
        println!("Adult");
    } else {
        println!("Minor");
    }

    // if is an expression, so it can produce a value:
    let status = if age >= 18 { "adult" } else { "minor" };
    println!("{status}");
}
```

4.2 Loops: loop, while, for

loop, while, for

```
fn main() {
    // `loop` runs forever until you `break`
    let mut counter = 0;
    let result = loop {
        counter += 1;
        if counter == 10 {
            break counter * 2; // break can return a value!
        }
    };
    println!("result = {result}");

    // `while` — condition checked first
    let mut retries = 3;
    while retries > 0 {
        println!("retrying... {retries} left");
        retries -= 1;
    }

    // `for` — the idiomatic choice for iterating
    let ids = [101, 102, 103];
    for id in ids {
        println!("processing task {id}");
    }

    for i in 0..5 { // range, exclusive of 5
        println!("i = {i}");
    }
}
```

TIP

Prefer **for** loops over manual indexing wherever possible. Rust's *for* loop is bounds-checked at compile time via iterators, so you can never accidentally read past the end of an array like you might in C.

4.3 match — Rust's supercharged switch

`match` is one of Rust's most-used features. Unlike a switch statement, it is **exhaustive** — the compiler forces you to handle every possible case, which eliminates a huge class of "forgot to handle this case" bugs in backend logic (e.g. handling every possible HTTP status or database result).

match expression

```
fn describe_status(code: u16) -> &'static str {
    match code {
        200 => "OK",
        201 => "Created",
        400 => "Bad Request",
        401 | 403 => "Unauthorized", // multiple patterns
        404 => "Not Found",
        500..=599 => "Server Error", // range pattern
        _ => "Unknown", // catch-all, required for exhaustiveness
    }
}

fn main() {
    println!("{}", describe_status(404));
}
```

■ Practice

Write a function `grade(score: u8) -> char` that returns 'A' for 90+, 'B' for 80-89, 'C' for 70-79, and 'F' otherwise, using `match` with range patterns.

Rewrite one of your `for` loops as a `while` loop, and notice how much more bookkeeping code you need to write manually.

Ownership, Borrowing & Lifetimes

This is the single most important chapter in this course. Ownership is the feature that lets Rust guarantee memory safety **without** a garbage collector. Once this clicks, everything else about Rust — including why async and concurrency are so safe — will make sense.

5.1 The three ownership rules

- 1 Every value in Rust has exactly one **owner** (a variable).
- 2 When the owner goes out of scope, the value is **dropped** (its memory is freed) automatically.
- 3 Ownership can be **moved** to another variable, or **borrowed** temporarily — but a value can only ever have one owner at a time.

A move — s1 is invalidated

```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1; // ownership MOVES from s1 to s2
    // println!("{s1}"); // compile error! s1 is no longer valid
    println!("{s2}"); // this works
}
```

NOTE

This is very different from JavaScript, where `let s2 = s1` just copies a reference and both variables keep working. In Rust, assigning a **heap-allocated** value like `String` *moves* it — the old variable becomes unusable. This prevents two variables from both thinking they own (and later trying to free) the same memory.

5.2 Borrowing with references

Instead of moving ownership every time, you can **borrow** a value using `&`. This is what you will do constantly when passing data into functions — you usually don't want to give up ownership just to read something.

Borrowing instead of moving

```
fn calculate_length(s: &String) -> usize { // borrows, does not take ownership
    s.len()
}

fn main() {
    let text = String::from("backend engineer");
    let len = calculate_length(&text); // pass a reference
    println!("'{text}' is {len} characters long"); // text is still usable here!
}
```

5.3 Mutable references — one rule to remember

You can have **either** one mutable reference **or** any number of immutable references to a value at the same time — never both. This single rule is what eliminates data races at compile time.

Mutable borrowing

```
fn add_exclaim(s: &mut String) {
    s.push_str("!");
}

fn main() {
    let mut msg = String::from("deploy successful");
    add_exclaim(&mut msg);
    println!("{msg}"); // "deploy successful!"

    // let r1 = &mut msg;
    // let r2 = &mut msg; // compile error: cannot borrow `msg` as mutable more than
    once
}
```

5.4 The Stack, the Heap, and Copy types

Simple, fixed-size types like integers, floats, and bool implement the **Copy** trait — assigning them duplicates the value instead of moving it, because copying an integer is cheap. Heap types (String, Vec, and your own structs by default) do **not** implement Copy, because duplicating them could be expensive or semantically wrong.

Copy types behave differently from moves

```
fn main() {
    let a = 5;
    let b = a; // COPY, not a move — both a and b are valid
    println!("{a} {b}"); // works fine
}
```

5.5 Lifetimes — a brief, practical view

Lifetimes tell the compiler how long a reference is valid for. Most of the time, Rust infers lifetimes automatically. You mainly need to write them explicitly when a function returns a reference that is derived from more than one input reference.

Explicit lifetime annotation

```
// 'a says: the returned reference lives at least as long as both x and y
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("backend");
    let s2 = String::from("api");
    println!("{}", longest(&s1, &s2));
}
```

TIP

In real backend code with Axum/Tokio you will rarely write lifetime annotations by hand — most structs own their data (using `String`, `Vec`, etc.) specifically to sidestep lifetime complexity. Learn to recognize them, but don't worry about mastering every edge case yet.

■ Practice

Write a function `word_count(text: &str) -> usize` that borrows a string and returns the number of words (hint: use .split_whitespace().count()).`

Predict, then verify: what happens if you try to use a `String` after passing it (not a reference) into a function that doesn't return it?

Structs & Enums

6.1 Structs — your custom data types

A struct groups related data together, similar to a TypeScript interface or a Java class's fields. Structs are the backbone of how you'll model backend entities like ``User``, ``Task``, or ``Order``.

Defining and using a struct

```
struct Task {
    id: u32,
    title: String,
    is_done: bool,
}

fn main() {
    let task = Task {
        id: 1,
        title: String::from("Learn ownership"),
        is_done: false,
    };
    println!("Task #{}: {} (done: {})", task.id, task.title, task.is_done);
}
```

6.2 Methods with impl

You attach behavior to a struct using an **impl** block. `&self;` borrows the instance (like ``this`` in JS/TS), while associated functions like ``new`` (no `self`) act like static factory methods.

impl blocks: constructors and methods

```
struct Task {
    id: u32,
    title: String,
    is_done: bool,
}

impl Task {
    // Associated function — acts like a constructor
    fn new(id: u32, title: &str) -> Task {
        Task { id, title: title.to_string(), is_done: false }
    }

    // Method — borrows self immutably
    fn summary(&self) -> String {
        format!("[{}] {}", if self.is_done { "x" } else { " " }, self.title)
    }

    // Method — borrows self mutably to change state
    fn complete(&mut self) {
        self.is_done = true;
    }
}
```



```

}

fn main() {
    let mut task = Task::new(1, "Ship the API");
    task.complete();
    println!("{}", task.summary());
}

```

6.3 Enums — modeling one-of-several states

Enums in Rust are far more powerful than in most languages — each variant can carry its own data. This makes them perfect for representing things like request status, API responses, or task priority.

Enums with data attached to variants

```

enum TaskStatus {
    Todo,
    InProgress { assignee: String }, // variant carrying named data
    Done(u64), // variant carrying a timestamp
}

fn describe(status: &TaskStatus) -> String {
    match status {
        TaskStatus::Todo => "Not started".to_string(),
        TaskStatus::InProgress { assignee } => format!("In progress, owned by {assignee}"),
        TaskStatus::Done(ts) => format!("Completed at timestamp {ts}"),
    }
}

fn main() {
    let s = TaskStatus::InProgress { assignee: "Siddhant".to_string() };
    println!("{}", describe(&s));
}

```

6.4 Option — Rust has no null

Rust does not have **null**. Instead, anything that might be absent is wrapped in the built-in **Option<T>** enum, which is either **Some(value)** or **None**. The compiler forces you to handle the "nothing" case, eliminating null-pointer bugs entirely.

Option instead of null

```

fn find_task(tasks: &[Task], id: u32) -> Option<&Task> {
    for t in tasks {
        if t.id == id {
            return Some(t);
        }
    }
    None
}

fn main() {
    let tasks = vec![Task::new(1, "Write tests")];
}

```

```
match find_task(&tasks, 1) {  
    Some(t) => println!("Found: {}", t.summary()),  
    None => println!("No task with that id"),  
}  
}
```

■ Practice

Add a `priority: TaskPriority` field to `Task`, where `TaskPriority` is an enum with `Low`, `Medium`, `High` variants.

Write a method `is_overdue(&self, now: u64) -> bool` that uses `Option` for an optional due date.

Pattern Matching in Depth

7.1 if let — for when you only care about one case

if let

```
fn main() {
    let maybe_id: Option<u32> = Some(42);

    // Instead of a full match just for one case:
    if let Some(id) = maybe_id {
        println!("Got id: {id}");
    } else {
        println!("No id provided");
    }
}
```

7.2 while let

while let — drain a Vec

```
fn main() {
    let mut stack = vec![1, 2, 3];
    while let Some(top) = stack.pop() {
        println!("popped: {top}");
    }
}
```

7.3 Destructuring structs and tuples in match

Destructuring with match

```
struct Point { x: i32, y: i32 }

fn classify(p: &Point) -> &'static str {
    match p {
        Point { x: 0, y: 0 } => "origin",
        Point { x: 0, .. } => "on the y-axis",
        Point { y: 0, .. } => "on the x-axis",
        _ => "somewhere else",
    }
}
```

7.4 Guards — extra conditions on a pattern

Match guards

```
fn describe(n: i32) -> &'static str {
    match n {
        x if x < 0 => "negative",
```

```
0 => "zero",
x if x % 2 == 0 => "positive and even",
_ => "positive and odd",
}
}
```

7.5 The ? operator — a preview

You will see the ? operator constantly in backend code (database calls, JSON parsing, HTTP requests). It's essentially a shortcut for "if this is an error/None, return early with that error; otherwise, unwrap the value and keep going." Chapter 9 covers this fully — this is just a preview.

A first look at ?

```
fn parse_and_double(input: &str) -> Result<i32, std::num::ParseIntError> {
    let n: i32 = input.parse()?; // returns early if parse fails
    Ok(n * 2)
}
```

■ Practice

Write a function that takes `Option` and uses `if let` to print double the value, or "nothing here" if `None`.

Add a match guard to the `grade()` function from Chapter 4 to give bonus output for a perfect score of 100.

Collections: Vec, HashMap, String

8.1 Vec — growable lists

Vec basics + iterator chains

```
fn main() {
    let mut tasks: Vec<String> = Vec::new();
    tasks.push("Design schema".to_string());
    tasks.push("Write handlers".to_string());

    // or with the macro:
    let numbers = vec![1, 2, 3, 4, 5];

    for t in &tasks {
        println!("{}", t);
    }

    let doubled: Vec<i32> = numbers.iter().map(|n| n * 2).collect();
    let evens: Vec<i32> = numbers.iter().filter(|n| *n % 2 == 0).collect();
    println!("{doubled:?} {evens:?}");
}
```

NOTE

`.iter().map(...).filter(...).collect()` chains are extremely common in Rust backend code — they read almost exactly like JavaScript's `.map()` / `.filter()`, but Rust checks the types of every step at compile time.

8.2 HashMap — key/value storage

HashMap basics + the entry API

```
use std::collections::HashMap;

fn main() {
    let mut users: HashMap<u32, String> = HashMap::new();
    users.insert(1, "Siddhant".to_string());
    users.insert(2, "Amit".to_string());

    match users.get(&1) {
        Some(name) => println!("User 1 is {name}"),
        None => println!("No such user"),
    }

    // entry API — very useful for counters / caches
    let mut visit_counts: HashMap<&str, u32> = HashMap::new();
    for route in ["/tasks", "/tasks", "/users"] {
        *visit_counts.entry(route).or_insert(0) += 1;
    }
    println!("{visit_counts:?}");
}
```

8.3 Strings in depth

String manipulation

```
fn main() {
    let mut s = String::from("Hello");
    s.push_str(", backend!");
    s.push('!');

    let combined = format!("{s} ({s} chars)", s.len());
    println!("{combined}");

    // slicing (careful: byte-indexed, not always safe with non-ASCII)
    let hello = &s[0..5];
    println!("{hello}");

    // iterating words/chars
    for word in s.split_whitespace() {
        println!("word: {word}");
    }
}
```

8.4 Choosing the right collection

You need to...	Use
Ordered list of items, possibly duplicates	Vec
Fast lookup by a key (like a JS object/dict)	HashMap
Unique items only, no order guarantee	HashSet
Sorted key-value pairs	BTreeMap
First-in-first-out queue	VecDeque

■ Practice — extend TaskFlow

Create a `Vec` with 3-4 sample tasks (reuse the struct from Chapter 6).

Use `.iter().filter()` to get only tasks where `is\_done == false`.

Build a `HashMap` keyed by task id, and write a function to look up a task by id in $O(1)$.

Error Handling

Backend services fail constantly — bad input, a database being down, a network timeout. Rust treats error handling as a first-class citizen instead of an afterthought. There is no exception system; errors are ordinary values you must explicitly handle.

9.1 Result

Any operation that can fail returns a **Result<T, E>** — either **Ok(value)** or **Err(error)**. This is the backend equivalent of a try/catch, but the compiler enforces that you handle failure.

Result basics

```
use std::num::ParseIntError;

fn parse_port(input: &str) -> Result<u16, ParseIntError> {
    input.parse::<u16>()
}

fn main() {
    match parse_port("8080") {
        Ok(port) => println!("Listening on port {port}"),
        Err(e) => println!("Invalid port: {e}"),
    }
}
```

9.2 The ? operator, fully explained

Chaining fallible operations with ?

```
fn parse_two_ports(a: &str, b: &str) -> Result<(u16, u16), ParseIntError> {
    let port_a = a.parse::<u16>()?; // if Err, returns immediately from parse_two_ports
    let port_b = b.parse::<u16>()?;
    Ok((port_a, port_b))
}
```

NOTE

The ? operator only works inside a function that itself returns Result (or Option). It automatically converts compatible error types too, which is why backend code often defines one central **AppError** enum — so every ? in a handler function can bubble up into it. We build exactly this in Chapter 17.

9.3 unwrap, expect, and why you should avoid them in production

unwrap / expect — fine for prototypes and tests, risky in handlers

```
fn main() {
    let port: u16 = "8080".parse().unwrap(); // panics on Err — avoid in real handlers
    let port2: u16 = "8080".parse().expect("bad port config"); // panics with a custom message
    println!("{port} {port2}");
}
```

```
}
```

WARNING

`unwrap()` crashes the whole thread if the `Result` is an `Err`. In a web server, an unhandled panic in a request handler can take down that request (or worse) — always prefer proper ``match`/`?`` handling in real backend code.

9.4 Defining your own error types

Custom error enum with Display

```
use std::fmt;

#[derive(Debug)]
enum TaskError {
    NotFound(u32),
    InvalidTitle,
}

impl fmt::Display for TaskError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            TaskError::NotFound(id) => write!(f, "task {id} not found"),
            TaskError::InvalidTitle => write!(f, "title cannot be empty"),
        }
    }
}

fn create_task(title: &str) -> Result<String, TaskError> {
    if title.trim().is_empty() {
        return Err(TaskError::InvalidTitle);
    }
    Ok(title.to_string())
}
```

9.5 The `thiserror` and `anyhow` crates

In real projects you rarely write `Display` impls by hand. Two crates dominate: **`thiserror`** (for defining clean library-style error enums) and **`anyhow`** (for quickly propagating errors in application code, especially prototypes).

`thiserror` — same error enum, for less boilerplate

```
use thiserror::Error;

#[derive(Debug, Error)]
enum TaskError {
    #[error("task {0} not found")]
    NotFound(u32),
    #[error("title cannot be empty")]
    InvalidTitle,
}
```


■ **Practice**

Rewrite `find_task` from Chapter 6 to return `Result<&Task, TaskError>` instead of `Option<&Task>`.

Add `cargo add thiserror` to your taskflow project and convert one hand-written error enum to use it.

Traits & Generics

10.1 Generics — write once, use with any type

A generic function that works for numbers AND strings

```
fn largest<T: PartialOrd + Copy>(list: &[T]) -> T {
    let mut largest = list[0];
    for &item in list {
        if item > largest {
            largest = item;
        }
    }
    largest
}

fn main() {
    let numbers = vec![10, 25, 3, 47, 8];
    let words = vec!["pear", "apple", "zebra"];
    println!("{}", largest(&numbers), largest(&words));
}
```

10.2 Traits — shared behavior (like interfaces)

A trait defines a set of methods a type must implement — very similar to a TypeScript interface, but traits can also provide default method implementations.

Defining and implementing a trait

```
trait Summarize {
    fn summary(&self) -> String;

    // default implementation – types can override or use as-is
    fn short_summary(&self) -> String {
        let full = self.summary();
        format!("{}", ..full[..full.len().min(20)])
    }
}

struct Task { title: String }

impl Summarize for Task {
    fn summary(&self) -> String {
        format!("Task: {}", self.title)
    }
}

fn main() {
    let t = Task { title: "Refactor the auth module".to_string() };
    println!("{}", t.summary());
}
```

10.3 Trait bounds — generic functions constrained by traits

impl Trait vs explicit generic bound — two equivalent styles

```
fn print_summary(item: &impl Summarize) {
    println!("{}", item.summary());
}

// equivalent, more explicit form using a trait bound:
fn print_summary_v2<T: Summarize>(item: &T) {
    println!("{}", item.summary());
}
```

10.4 Common derivable traits

Rust can auto-generate several traits for your structs/enums with `#[derive(...)]`. You will use these constantly, especially **Serialize/Deserialize** from the **serde** crate for JSON in web APIs.

Trait	What it enables
Debug	<code>{:?}</code> formatting for <code>println!/logs</code>
Clone	<code>.clone()</code> — explicit deep copy
PartialEq	<code>==</code> and <code>!=</code> comparisons
Default	<code>Type::default()</code> for a sensible empty value
Serialize / Deserialize (serde)	Convert struct <code><-></code> JSON automatically

This one derive block is what makes Task work as a JSON API response later

```
use serde::{Serialize, Deserialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
struct Task {
    id: u32,
    title: String,
    is_done: bool,
}
```

10.5 Trait objects — dynamic dispatch

Box — a heap-allocated value of any type implementing Summarize

```
fn print_all(items: &[Box<dyn Summarize>]) {
    for item in items {
        println!("{}", item.summary());
    }
}
```

TIP

Use generics (**impl Trait** / **<T: Trait>**) when the compiler can know the concrete type at compile time — it's faster ("static dispatch"). Use **Box<dyn Trait>** when you need a collection of different types behind one interface, e.g. a list of different notification handlers.

■ Practice

Add a `Priced` trait with a `price(&self) -> f64` method, and implement it for two different structs. Add `#[derive(Serialize, Deserialize)]` to your Task struct — you'll need `cargo add serde --features derive`.

Modules, Crates & Cargo Workspaces

11.1 Modules — organizing code within a crate

Splitting code across files with `mod`

```
// src/main.rs
mod tasks; // loads src/tasks.rs (or src/tasks/mod.rs)

fn main() {
    let t = tasks::Task::new(1, "Learn modules");
    println!("{}", t.title);
}

// src/tasks.rs
pub struct Task {
    pub id: u32,
    pub title: String,
}

impl Task {
    pub fn new(id: u32, title: &str) -> Self {
        Self { id, title: title.to_string() }
    }
}
```

NOTE

Everything in Rust is **private by default**. You must mark structs, fields, and functions with **pub** to expose them outside their module — the opposite default of most languages, and another safety-first design choice.

11.2 A realistic backend folder layout

This exact layout is what we build in Chapters 14-19.

```
taskflow/
├── Cargo.toml
├── src/
│   ├── main.rs # starts the server
│   ├── routes/ # HTTP route handlers
│   │   ├── mod.rs
│   │   └── tasks.rs
│   ├── models/ # data structures (Task, User...)
│   │   ├── mod.rs
│   │   └── db/ # database connection + queries
│   │       ├── mod.rs
│   └── errors.rs # the shared AppError type
```

11.3 Crates: binaries vs libraries

- A **binary crate** has a `main.rs` and compiles to an executable — your server.
- A **library crate** has a `lib.rs` and exposes reusable code — useful once your project grows and you want to share logic between a server binary and a CLI tool, for example.
- Crates published to **crates.io** (Rust's package registry, like npm) are what you pull in with `cargo add`.

11.4 Cargo workspaces

A workspace lets multiple related crates share one `Cargo.lock` and target directory — useful once your backend splits into, say, an API server crate and a shared "core" library crate.

A two-crate workspace

```
# Cargo.toml at the workspace root
[workspace]
members = ["api", "core"]
```

11.5 Useful crates.io packages for backend work

Crate	Purpose
tokio	Async runtime — powers almost every async Rust backend
axum	Web framework for building HTTP APIs
serde / serde_json	Serialize & deserialize structs to/from JSON
sqlx	Async, compile-time checked SQL queries
tracing	Structured logging for backend services
jsonwebtoken	Encode/decode JWT auth tokens
dotenvy	Load environment variables from a .env file

■ Practice

Reorganize your taskflow project: move the `Task` struct into `src/models/mod.rs` and import it in `main.rs`.

Run `cargo add tokio axum serde --features tokio/full,serde/derive` to prep for the next chapters.

Concurrency with Threads

Before `async` (Chapter 13), it's worth understanding Rust's OS-thread based concurrency — the foundation everything else is built on, and still useful for CPU-heavy backend work like image processing or batch computation.

12.1 Spawning threads

Basic threading

```
use std::thread;
use std::time::Duration;

fn main() {
    let handle = thread::spawn(|| {
        for i in 1..=3 {
            println!("worker: {i}");
            thread::sleep(Duration::from_millis(50));
        }
    });

    println!("main thread continues immediately");
    handle.join().unwrap(); // wait for the thread to finish
}
```

12.2 Sharing data safely: `Arc`

This is the pattern you will see constantly in real backend code for any shared, mutable state (an in-memory cache, a connection counter, etc). **Arc** (Atomic Reference Count) lets multiple threads share ownership; **Mutex** ensures only one thread mutates the data at a time.

`Arc` — the standard shared state pattern

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();
            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }
}
```

```
println!("Result: {}", *counter.lock().unwrap()); // always 10 – guaranteed by the compiler
}
```

TIP

In JavaScript this kind of shared counter across workers would need careful manual coordination. In Rust, the compiler literally will not let you access `counter` from a thread without going through the Mutex — a data race is a compile error, not a runtime bug you discover in production.

12.3 Channels — message passing between threads

mpsc channel — multiple producers, single consumer

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        for msg in ["job1 done", "job2 done", "job3 done"] {
            tx.send(msg.to_string()).unwrap();
        }
    });

    for received in rx {
        println!("main got: {received}");
    }
}
```

■ Practice

Spawn 5 threads that each simulate "processing a task" (sleep + print), then join them all. Use `Arc<>>` so each thread can push a completion message into a shared log.

Async/Await & Tokio

Every real backend spends most of its time **waiting** — for a database query, a network call, a disk read. Threads work, but they're relatively heavyweight (megabytes of stack each). Async lets a single thread juggle thousands of waiting tasks efficiently. This is how virtually every modern Rust web server, including the one we build later, handles concurrency.

13.1 async fn and .await

An async function must be awaited to actually run.

```
async fn fetch_user(id: u32) -> String {
    // imagine this is a real network/database call
    format!("user-{{id}}")
}

async fn run() {
    let user = fetch_user(1).await; // pauses here without blocking the thread
    println!("{user}");
}
```

NOTE

Calling an async function does **not** run it — it returns a lazy `Future``. Nothing happens until something `.await`s` it (or a runtime polls it). This trips up almost everyone the first time.

13.2 Tokio — the async runtime

Rust's standard library deliberately does not ship an async runtime — you choose one. **Tokio** is the de-facto standard for backend work, and it's what Axum (our web framework) is built on.

#[tokio::main] sets up the runtime for you.

```
// Cargo.toml: tokio = { version = "1", features = ["full"] }

#[tokio::main]
async fn main() {
    println!("server starting...");
    let user = fetch_user(1).await;
    println!("loaded {user}");
}

async fn fetch_user(id: u32) -> String {
    format!("user-{{id}}")
}
```

13.3 Running tasks concurrently

tokio::join! runs multiple futures concurrently

```

use tokio::time::{sleep, Duration};

async fn slow_task(name: &str, ms: u64) -> String {
    sleep(Duration::from_millis(ms)).await;
    format!("{name} finished")
}

#[tokio::main]
async fn main() {
    // sequential – takes ~300ms total
    let a = slow_task("A", 150).await;
    let b = slow_task("B", 150).await;

    // concurrent – takes ~150ms total, both run at once
    let (c, d) = tokio::join!(slow_task("C", 150), slow_task("D", 150));

    println!("{a} {b} {c} {d}");
}

```

13.4 Spawning independent tasks

tokio::spawn – fire off an independent async task

```

#[tokio::main]
async fn main() {
    let handle = tokio::spawn(async {
        slow_task("background job", 100).await
    });

    println!("doing other work while the job runs...");
    let result = handle.await.unwrap();
    println!("{result}");
}

```

Concept	Analogy for a JS/TS developer
async fn	an async function, like in JavaScript
.await	the same idea as JS `await`
Future	conceptually similar to a JS Promise, but lazy (does nothing until polled)
tokio::spawn	similar to firing an async task without awaiting it immediately
tokio::join!	similar to Promise.all()

■ Practice

Add `tokio` to a fresh project with the `full` feature and write an async main that awaits two simulated network calls using `tokio::join!`.

Use `tokio::spawn` to run a background task and print a message from `main` before awaiting its result.

Building a Web Backend with Axum

Axum is built by the Tokio team and is currently one of the most popular Rust web frameworks — fast, type-safe, and integrates directly with the async ecosystem you just learned. From here on, every chapter builds directly onto the TaskFlow API.

14.1 Project setup

Terminal

```
cargo new taskflow
cd taskflow
cargo add axum tokio --features tokio/full
cargo add serde --features derive
cargo add serde_json
```

14.2 A minimal server

src/main.rs — the whole server

```
use axum::{routing::get, Router};

async fn health_check() -> &'static str {
    "OK"
}

#[tokio::main]
async fn main() {
    let app = Router::new().route("/health", get(health_check));

    let listener = tokio::net::TcpListener::bind("0.0.0.0:3000").await.unwrap();
    println!("listening on http://localhost:3000");
    axum::serve(listener, app).await.unwrap();
}
```

Run it and test with curl:

Terminal

```
cargo run
curl http://localhost:3000/health
```

14.3 JSON request/response with serde

Handlers that automatically parse and return JSON

```
use axum::{routing::{get, post}, Router, Json, extract::Path};
use serde::{Deserialize, Serialize};

#[derive(Serialize, Clone)]
struct Task {
    id: u32,
```

```

        title: String,
        is_done: bool,
    }

    #[derive(Deserialize)]
    struct CreateTask {
        title: String,
    }

    async fn list_tasks() -> Json<Vec<Task>> {
        Json(vec![
            Task { id: 1, title: "Learn Axum".into(), is_done: false },
        ])
    }

    async fn create_task(Json(payload): Json<CreateTask>) -> Json<Task> {
        Json(Task { id: 2, title: payload.title, is_done: false })
    }

    async fn get_task(Path(id): Path<u32>) -> Json<Task> {
        Json(Task { id, title: "Sample task".into(), is_done: false })
    }

    #[tokio::main]
    async fn main() {
        let app = Router::new()
            .route("/tasks", get(list_tasks).post(create_task))
            .route("/tasks/:id", get(get_task));

        let listener = tokio::net::TcpListener::bind("0.0.0.0:3000").await.unwrap();
        axum::serve(listener, app).await.unwrap();
    }

```

TIP

Notice how little "plumbing" code exists — Axum uses **extractors** like ``Json`` and ``Path`` as function parameters, and it automatically parses the request body/URL and produces a 400 error if the shape doesn't match your struct. Compare this to manually parsing ``req.body`` in Express.

14.4 Shared application state

Real handlers need access to shared resources — a database pool, a config, an in-memory cache. Axum's **State** extractor combined with **Arc** (from Chapter 12!) is the standard pattern.

Shared, thread-safe state across every request

```

use axum::extract::State;
use std::sync::{Arc, Mutex};

#[derive(Clone)]
struct AppState {
    tasks: Arc<Mutex<Vec<Task>>>,
}

async fn list_tasks(State(state): State<AppState>) -> Json<Vec<Task>> {

```

```

        let tasks = state.tasks.lock().unwrap();
        Json(tasks.clone())
    }

    #[tokio::main]
    async fn main() {
        let state = AppState { tasks: Arc::new(Mutex::new(vec![])) };

        let app = Router::new()
            .route("/tasks", get(list_tasks))
            .with_state(state);

        let listener = tokio::net::TcpListener::bind("0.0.0.0:3000").await.unwrap();
        axum::serve(listener, app).await.unwrap();
    }

```

14.5 Route summary for TaskFlow

Method & Path	Purpose
GET /health	Health check for uptime monitors
GET /tasks	List all tasks
POST /tasks	Create a task
GET /tasks/:id	Get one task
PUT /tasks/:id	Update a task
DELETE /tasks/:id	Delete a task
POST /auth/register	Create a user account
POST /auth/login	Get a JWT token

■ Practice — extend TaskFlow

Implement `PUT /tasks/:id` and `DELETE /tasks/:id` handlers against the in-memory `AppState`. Test all your routes with curl or a tool like Postman/Insomnia/HTTPIe.

Databases with SQLx & PostgreSQL

An in-memory Vec disappears every time the server restarts — real backends need a database. **SQLx** is an async SQL toolkit that checks your queries against the real database schema at **compile time**, so a typo in a column name is a compile error, not a 3am production bug.

15.1 Setup

Adding SQLx to a .env for local config

```
cargo add sqlx --features runtime-tokio,postgres,macros,chrono,uuid
cargo add dotenvy

# .env
DATABASE_URL=postgres://postgres:password@localhost:5432/taskflow
```

15.2 Migrations

Terminal — sets up a migrations/ folder

```
cargo install sqlx-cli --no-default-features --features postgres,rustls
sqlx database create
sqlx migrate add create_tasks_table
```

SQL migration file

```
-- migrations/xxxx_create_tasks_table.sql
CREATE TABLE tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title TEXT NOT NULL,
  is_done BOOLEAN NOT NULL DEFAULT FALSE,
  user_id UUID NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Terminal — applies the migration

```
sqlx migrate run
```

15.3 Connecting a pool and running queries

One pool, shared across all requests via AppState

```
use sqlx::postgres::{PgPool, PgPoolOptions};

async fn create_pool() -> PgPool {
  let url = std::env::var("DATABASE_URL").expect("DATABASE_URL not set");
  PgPoolOptions::new()
    .max_connections(5)
    .connect(&url)
    .await
}
```

```

        .expect("failed to connect to database")
    }

```

Type-checked queries with `query_as`

```

use uuid::Uuid;
use serde::Serialize;

#[derive(Serialize, sqlx::FromRow)]
struct Task {
    id: Uuid,
    title: String,
    is_done: bool,
}

async fn list_tasks(pool: &PgPool) -> Result<Vec<Task>, sqlx::Error> {
    sqlx::query_as::<_, Task>("SELECT id, title, is_done FROM tasks ORDER BY created_at
DESC")
        .fetch_all(pool)
        .await
}

async fn create_task(pool: &PgPool, title: &str, user_id: Uuid) -> Result<Task,
sqlx::Error> {
    sqlx::query_as::<_, Task>(
        "INSERT INTO tasks (title, user_id) VALUES ($1, $2) RETURNING id, title,
is_done"
    )
        .bind(title)
        .bind(user_id)
        .fetch_one(pool)
        .await
}

```

NOTE

SQLx's ``query!`` and ``query_as!`` macros (with the trailing `!``) go one step further — they connect to your real database *while compiling* and verify your SQL and Rust types match exactly. We use the non-macro form above so the examples compile without a live database, but reach for the macro form in your real project once your database is running.

15.4 Wiring the database pool into Axum

Handlers now *query real Postgres* instead of a *Vec* in memory.

```

#[derive(Clone)]
struct AppState {
    db: PgPool,
}

async fn list_tasks_handler(State(state): State<AppState>) -> Json<Vec<Task>> {
    let tasks = list_tasks(&state.db).await.unwrap_or_default();
    Json(tasks)
}

```

■ Practice

Install PostgreSQL locally (or use Docker: ``docker run -e POSTGRES_PASSWORD=pass -p 5432:5432 postgres``).

Write and run a migration for a ``users`` table with ``id``, ``email``, ``password_hash``, ``created_at``.

Rewire your TaskFlow handlers from Chapter 14 to use ``PgPool`` instead of the in-memory `Vec`.

Authentication with JWT

Most backends need to know **who** is calling an endpoint. JSON Web Tokens (JWT) are the standard approach for stateless APIs: the server signs a token containing the user's identity, and the client sends it back on every request.

16.1 Password hashing

Never store plain-text passwords. **argon2** is the current best-practice hashing algorithm for passwords.

Terminal

```
cargo add argon2
cargo add rand
```

Hashing and verifying passwords

```
use argon2::{Argon2, PasswordHasher, PasswordVerifier, PasswordHash};
use argon2::password_hash::{SaltString, rand_core::OsRng};

fn hash_password(plain: &str) -> String {
    let salt = SaltString::generate(&mut OsRng);
    Argon2::default()
        .hash_password(plain.as_bytes(), &salt)
        .unwrap()
        .to_string()
}

fn verify_password(plain: &str, hash: &str) -> bool {
    let parsed = PasswordHash::new(hash).unwrap();
    Argon2::default().verify_password(plain.as_bytes(), &parsed).is_ok()
}
```

16.2 Issuing a JWT on login

Terminal

```
cargo add jsonwebtoken
cargo add chrono --features serde
```

Signing a token that expires in 24 hours

```
use jsonwebtoken::{encode, Header, EncodingKey};
use serde::{Serialize, Deserialize};
use chrono::{Utc, Duration};

#[derive(Serialize, Deserialize)]
struct Claims {
    sub: String, // user id
    exp: usize, // expiry, unix timestamp
}
```

```
fn create_jwt(user_id: &str, secret: &str) -> String {
    let expiration = Utc::now() + Duration::hours(24);
    let claims = Claims { sub: user_id.to_string(), exp: expiration.timestamp() as usize };
    encode(&Header::default(), &claims,
    &EncodingKey::from_secret(secret.as_bytes())).unwrap()
}
```

16.3 Verifying tokens with an Axum middleware/extractor

A custom extractor = automatic auth checking for any route that needs it

```
use axum::{
    async_trait, extract::FromRequestParts, http::{request::Parts, StatusCode},
};
use jsonwebtoken::{decode, DecodingKey, Validation};

struct AuthUser { user_id: String }

#[async_trait]
impl<S: Send + Sync> FromRequestParts<S> for AuthUser {
    type Rejection = (StatusCode, &'static str);

    async fn from_request_parts(parts: &mut Parts, _state: &S) -> Result<Self,
    Self::Rejection> {
        let header = parts.headers.get("Authorization")
            .and_then(|h| h.to_str().ok())
            .ok_or((StatusCode::UNAUTHORIZED, "missing token"))?;

        let token = header.strip_prefix("Bearer ")
            .ok_or((StatusCode::UNAUTHORIZED, "invalid auth header"))?;

        let secret = std::env::var("JWT_SECRET").unwrap();
        let data = decode::<Claims>(
            token, &DecodingKey::from_secret(secret.as_bytes()), &Validation::default()
        ).map_err(|_| (StatusCode::UNAUTHORIZED, "invalid token"))?;

        Ok(AuthUser { user_id: data.claims.sub })
    }
}

// Now any handler can simply request an AuthUser and Axum verifies the JWT
// automatically:
async fn my_tasks(user: AuthUser) -> String {
    format!("tasks for user {}", user.user_id)
}
```

TIP

This is the same pattern as an Express `authMiddleware`, but the type system enforces it: if a handler declares `user: AuthUser` as a parameter, it is **impossible** to reach that handler's code without a valid token, because Axum won't even call the function otherwise.

■ **Practice**

Implement `POST /auth/register` (hash + store password) and `POST /auth/login` (verify + issue JWT).

Protect `GET /tasks` so it requires a valid `AuthUser`, and scope results to that user's own tasks.

Middleware, Logging & Centralized Error Handling

17.1 A single AppError for the whole API

Instead of every handler returning a different error shape, define one **AppError** enum that implements Axum's **IntoResponse** trait. Every ? in every handler can then bubble up into it automatically, and every failure gets converted into a consistent JSON error response.

One error type to rule every handler

```
use axum::{response::{IntoResponse, Response}, http::StatusCode, Json};
use serde_json::json;

#[derive(Debug, thiserror::Error)]
enum AppError {
    #[error("not found")]
    NotFound,
    #[error("unauthorized")]
    Unauthorized,
    #[error("database error: {0}")]
    Database(#[from] sqlx::Error),
    #[error("validation error: {0}")]
    Validation(String),
}

impl IntoResponse for AppError {
    fn into_response(self) -> Response {
        let (status, message) = match &self {
            AppError::NotFound => (StatusCode::NOT_FOUND, self.to_string()),
            AppError::Unauthorized => (StatusCode::UNAUTHORIZED, self.to_string()),
            AppError::Validation(_) => (StatusCode::BAD_REQUEST, self.to_string()),
            AppError::Database(_) => (StatusCode::INTERNAL_SERVER_ERROR, "internal
error".to_string()),
        };
        (status, Json(json!({ "error": message }))).into_response()
    }
}
```

Handlers become short and readable — the ? operator does the heavy lifting

```
async fn get_task(
    State(state): State<AppState>,
    Path(id): Path<Uuid>,
) -> Result<Json<Task>, AppError> {
    let task = sqlx::query_as!(<_, Task>("SELECT * FROM tasks WHERE id = $1")
        .bind(id)
        .fetch_optional(&state.db)
        .await? // sqlx::Error automatically becomes AppError::Database
        .ok_or(AppError::NotFound)?;

    Ok(Json(task))
}
```

17.2 Structured logging with tracing

Terminal

```
cargo add tracing tracing-subscriber --features tracing-subscriber/env-filter
```

tracing gives you structured, filterable logs — essential once you're debugging production

```
use tracing::{info, warn, error};

#[tokio::main]
async fn main() {
    tracing_subscriber::fmt()
        .with_env_filter("taskflow=debug,tower_http=debug")
        .init();

    info!("starting taskflow server");
    // ...
}

async fn create_task_handler(/* ... */) {
    info!(title = %"new task", "creating task");
    // on failure:
    // error!(error = %e, "failed to insert task");
}
```

17.3 Middleware: request logging, CORS, timeouts

Axum is built on **tower**, an ecosystem of composable middleware. You rarely write middleware from scratch — instead you layer pre-built ones from **tower-http** onto your router.

Terminal

```
cargo add tower-http --features trace,cors,timeout
```

Layering middleware onto the router

```
use tower_http::{trace::TraceLayer, cors::CorsLayer, timeout::TimeoutLayer};
use std::time::Duration;

let app = Router::new()
    .route("/tasks", get(list_tasks_handler))
    .layer(TraceLayer::new_for_http()) // logs every request
    .layer(CorsLayer::permissive()) // allows cross-origin requests
    .layer(TimeoutLayer::new(Duration::from_secs(10)))
    .with_state(state);
```

17.4 Input validation

Terminal

```
cargo add validator --features derive
```

```

use validator::Validate;

#[derive(Deserialize, Validate)]
struct CreateTask {
    #[validate(length(min = 1, max = 200))]
    title: String,
}

async fn create_task_handler(
    Json(payload): Json<CreateTask>,
) -> Result<Json<Task>, AppError> {
    payload.validate().map_err(|e| AppError::Validation(e.to_string()))?;
    // ... insert into database
    # unimplemented!()
}

```

■ Practice

Replace every `.unwrap()` in your TaskFlow handlers with proper `AppError` propagation via `?`. Add `TraceLayer` and confirm you can see structured request logs in your terminal when calling routes.

Add `#[validate(email)]` to a `RegisterUser` struct's email field.

Testing in Rust

18.1 Unit tests — built into the language

Rust's testing framework ships in the standard library — no separate test runner to install. Tests live alongside your code inside a `#[cfg(test)]` module.

cargo test runs every `#[test]` function automatically.

```
fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn adds_two_numbers() {
        assert_eq!(add(2, 3), 5);
    }

    #[test]
    fn adds_negative_numbers() {
        assert_eq!(add(-2, -3), -5);
    }
}
```

Terminal

```
cargo test
```

18.2 Testing Result-returning functions

Tests can themselves return `Result` — a failing `2` fails the test.

```
#[test]
fn rejects_empty_title() {
    let result = create_task("");
    assert!(result.is_err());
}

#[test]
fn accepts_valid_title() -> Result<(), TaskError> {
    let title = create_task("Write docs")?;
    assert_eq!(title, "Write docs");
    Ok(())
}
```

18.3 Async tests

#[tokio::test] is the async equivalent of #[test]

```
#[tokio::test]
async fn fetches_a_task() {
    let pool = create_test_pool().await;
    let task = create_task(&pool, "test task", test_user_id()).await.unwrap();
    assert_eq!(task.title, "test task");
}
```

18.4 Integration tests for your API

Integration tests live in a top-level `tests/`` folder and exercise your app the way a real client would — through actual HTTP requests, not internal function calls.

tests/health_check.rs — a real HTTP round trip against your router, no network socket needed

```
// tests/health_check.rs
use axum::body::Body;
use axum::http::Request;
use tower::ServiceExt; // for `oneshot`

#[tokio::test]
async fn health_check_returns_ok() {
    let app = taskflow::build_app().await; // your Router-building function

    let response = app
        .oneshot(Request::builder().uri("/health").body(Body::empty()).unwrap())
        .await
        .unwrap();

    assert_eq!(response.status(), 200);
}
```

TIP

For this to work cleanly, structure `main.rs` so route-building lives in a reusable function (e.g. `pub async fn build_app() -> Router`) that both `main()` and your tests can call. This is the same principle as exporting an Express `app` object separately from `app.listen()` for supertest.

18.5 Useful assertion & test crates

Crate	Purpose
assert_matches / pretty_assertions	Nicer diff output on failing assertions
mockall	Generate mock implementations of traits for isolated unit tests
fake	Generate realistic fake test data (names, emails, etc.)
sqlx::test	Spins up an isolated test database transaction per test

■ Practice

Write unit tests for your ``AppError`` → HTTP status mapping logic.

Write one integration test per route in TaskFlow, checking both success and failure status codes.

Run ``cargo test`` and get every test passing before moving to Chapter 19.

Building TaskFlow End-to-End

This chapter assembles everything from Chapters 1-18 into one complete, working backend service: **TaskFlow**, a multi-user task management REST API with authentication and PostgreSQL storage. Follow it top to bottom, or use it as a reference architecture for your own projects.

19.1 Final project structure

The complete layout

```
taskflow/
  ■■■ Cargo.toml
  ■■■ .env
  ■■■ migrations/
  ■ ■■■ 0001_create_users.sql
  ■ ■■■ 0002_create_tasks.sql
  ■■■ src/
  ■ ■■■ main.rs
  ■ ■■■ state.rs # AppState (db pool, jwt secret)
  ■ ■■■ errors.rs # AppError + IntoResponse
  ■ ■■■ auth/
  ■ ■ ■■■ mod.rs
  ■ ■ ■■■ jwt.rs # create_jwt / AuthUser extractor
  ■ ■ ■■■ password.rs # hash_password / verify_password
  ■ ■■■ models/
  ■ ■ ■■■ mod.rs
  ■ ■ ■■■ user.rs
  ■ ■ ■■■ task.rs
  ■ ■■■ routes/
  ■ ■■■ mod.rs
  ■ ■■■ auth.rs # /auth/register, /auth/login
  ■ ■■■ tasks.rs # /tasks CRUD
  ■■■ tests/
  ■■■ api.rs
```

19.2 Cargo.toml

Cargo.toml — every dependency this project needs

```
[package]
name = "taskflow"
version = "0.1.0"
edition = "2021"

[dependencies]
axum = "0.7"
tokio = { version = "1", features = ["full"] }
tower-http = { version = "0.5", features = ["trace", "cors"] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
sqlx = { version = "0.7", features = ["runtime-tokio", "postgres", "macros", "uuid", "chrono"] }
```

```

uuid = { version = "1", features = ["v4", "serde"] }
chrono = { version = "0.4", features = ["serde"] }
jsonwebtoken = "9"
argon2 = "0.5"
thiserror = "1"
dotenvy = "0.15"
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter"] }
validator = { version = "0.18", features = ["derive"] }

```

19.3 Database schema

Two tables: users and their tasks

```

-- migrations/0001_create_users.sql
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- migrations/0002_create_tasks.sql
CREATE TABLE tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title TEXT NOT NULL,
    is_done BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

19.4 Application state

src/state.rs

```

// src/state.rs
use sqlx::PgPool;

#[derive(Clone)]
pub struct AppState {
    pub db: PgPool,
    pub jwt_secret: String,
}

```

19.5 Models

src/models/task.rs — kept separate from database rows on purpose (never serialize password_hash!)

```

// src/models/task.rs
use serde::Serialize;
use uuid::Uuid;
use chrono::{DateTime, Utc};

```

```

#[derive(Serialize, sqlx::FromRow)]
pub struct Task {
    pub id: Uuid,
    pub title: String,
    pub is_done: bool,
    pub created_at: DateTime<Utc>,
}

// src/models/user.rs
use serde::Serialize;

#[derive(sqlx::FromRow)]
pub struct User {
    pub id: Uuid,
    pub email: String,
    pub password_hash: String,
}

#[derive(Serialize)]
pub struct UserPublic {
    pub id: Uuid,
    pub email: String,
}

```

19.6 Auth routes

src/routes/auth.rs — register & login, tying together Ch 15 (SQLx) + Ch 16 (JWT/argon2) + Ch 17 (AppError)

```

// src/routes/auth.rs
use axum::{extract::State, Json};
use serde::{Deserialize, Serialize};
use validator::Validate;
use crate::{state::AppState, errors::AppError, models::user::User};
use crate::auth::{password::{hash_password, verify_password}, jwt::create_jwt};

#[derive(Deserialize, Validate)]
pub struct RegisterRequest {
    #[validate(email)]
    email: String,
    #[validate(length(min = 8))]
    password: String,
}

#[derive(Serialize)]
pub struct AuthResponse {
    token: String,
}

pub async fn register(
    State(state): State<AppState>,
    Json(req): Json<RegisterRequest>,
) -> Result<Json<AuthResponse>, AppError> {
    req.validate().map_err(|e| AppError::Validation(e.to_string()))?;
    let hash = hash_password(&req.password);
}

```

```

        let user: User = sqlx::query_as(
            "INSERT INTO users (email, password_hash) VALUES ($1, $2) RETURNING id, email,
password_hash"
        )
        .bind(&req.email)
        .bind(&hash)
        .fetch_one(&state.db)
        .await?;

        let token = create_jwt(&user.id.to_string(), &state.jwt_secret);
        Ok(Json(AuthResponse { token }))
    }

#[derive(Deserialize)]
pub struct LoginRequest { email: String, password: String }

pub async fn login(
    State(state): State<AppState>,
    Json(req): Json<LoginRequest>,
) -> Result<Json<AuthResponse>, AppError> {
    let user: User = sqlx::query_as("SELECT * FROM users WHERE email = $1")
        .bind(&req.email)
        .fetch_optional(&state.db)
        .await?
        .ok_or(AppError::Unauthorized)?;

    if !verify_password(&req.password, &user.password_hash) {
        return Err(AppError::Unauthorized);
    }

    let token = create_jwt(&user.id.to_string(), &state.jwt_secret);
    Ok(Json(AuthResponse { token }))
}

```

19.7 Task routes

src/routes/tasks.rs — every query is scoped to the authenticated user

```

// src/routes/tasks.rs
use axum::{extract::{State, Path}, Json};
use serde::Deserialize;
use uuid::Uuid;
use crate::{state::AppState, errors::AppError, models::task::Task,
auth::jwt::AuthUser};

#[derive(Deserialize)]
pub struct CreateTask { title: String }

pub async fn list_tasks(
    State(state): State<AppState>,
    user: AuthUser,
) -> Result<Json<Vec<Task>>, AppError> {
    let tasks = sqlx::query_as::<_, Task>(
        "SELECT id, title, is_done, created_at FROM tasks WHERE user_id = $1 ORDER BY
created_at DESC"
    )
    .fetch(&state.db)
    .await?;
    Ok(Json(tasks))
}

```

```

    )
    .bind(Uuid::parse_str(&user.user_id).map_err(|_| AppError::Unauthorized?))
    .fetch_all(&state.db)
    .await?;

    Ok(Json(tasks))
}

pub async fn create_task(
    State(state): State<AppState>,
    user: AuthUser,
    Json(req): Json<CreateTask>,
) -> Result<Json<Task>, AppError> {
    if req.title.trim().is_empty() {
        return Err(AppError::Validation("title cannot be empty".into()));
    }
    let uid = Uuid::parse_str(&user.user_id).map_err(|_| AppError::Unauthorized?);

    let task = sqlx::query_as::<_, Task>(
        "INSERT INTO tasks (title, user_id) VALUES ($1, $2) RETURNING id, title,
is_done, created_at"
    )
    .bind(&req.title)
    .bind(uid)
    .fetch_one(&state.db)
    .await?;

    Ok(Json(task))
}

pub async fn delete_task(
    State(state): State<AppState>,
    user: AuthUser,
    Path(id): Path<Uuid>,
) -> Result<(), AppError> {
    let uid = Uuid::parse_str(&user.user_id).map_err(|_| AppError::Unauthorized?);
    let result = sqlx::query("DELETE FROM tasks WHERE id = $1 AND user_id = $2")
        .bind(id)
        .bind(uid)
        .execute(&state.db)
        .await?;

    if result.rows_affected() == 0 {
        return Err(AppError::NotFound);
    }
    Ok(())
}

```

19.7b Updating a task and adding pagination

Two extensions that come up in nearly every real list-based API: an update endpoint, and pagination so `GET /tasks` doesn't return an unbounded result set as a user's task list grows.

PUT /tasks/:id and paginated GET /tasks — clamp() is doing real defensive work here

```

// src/routes/tasks.rs (continued)
use axum::extract::Query;

#[derive(Deserialize)]
pub struct UpdateTask {
    title: Option<String>,
    is_done: Option<bool>,
}

pub async fn update_task(
    State(state): State<AppState>,
    user: AuthUser,
    Path(id): Path<Uuid>,
    Json(req): Json<UpdateTask>,
) -> Result<Json<Task>, AppError> {
    let uid = Uuid::parse_str(&user.user_id).map_err(|_| AppError::Unauthorized)?;

    // COALESCE keeps the existing value when the field wasn't provided in the request
    let task = sqlx::query_as::<_, Task>(
        "UPDATE tasks SET title = COALESCE($1, title), is_done = COALESCE($2, is_done)
        WHERE id = $3 AND user_id = $4
        RETURNING id, title, is_done, created_at"
    )
    .bind(req.title)
    .bind(req.is_done)
    .bind(id)
    .bind(uid)
    .fetch_optional(&state.db)
    .await?
    .ok_or(AppError::NotFound)?;

    Ok(Json(task))
}

#[derive(Deserialize)]
pub struct Pagination {
    #[serde(default = "default_limit")]
    limit: i64,
    #[serde(default)]
    offset: i64,
}

fn default_limit() -> i64 { 20 }

pub async fn list_tasks_paginated(
    State(state): State<AppState>,
    user: AuthUser,
    Query(page): Query<Pagination>,
) -> Result<Json<Vec<Task>>, AppError> {
    let uid = Uuid::parse_str(&user.user_id).map_err(|_| AppError::Unauthorized)?;
    let limit = page.limit.clamp(1, 100); // never let a client request an unbounded
    page size

    let tasks = sqlx::query_as::<_, Task>(
        "SELECT id, title, is_done, created_at FROM tasks
        WHERE user_id = $1 ORDER BY created_at DESC LIMIT $2 OFFSET $3"
    )
    .bind(uid)

```

```

        .bind(limit)
        .bind(page.offset)
        .fetch_all(&state.db)
        .await?;

    Ok(Json(tasks))
}

// GET /tasks?limit=10&offset=20

```

WARNING

Always clamp or cap client-supplied `limit` values. Without it, a client requesting `limit=999999999` could force a single query to load and serialize far more data than intended — a common, easy-to-miss source of accidental denial-of-service in list endpoints.

19.8 Wiring it all together in main.rs

src/main.rs — the complete server, ~50 lines of wiring on top of everything you've built

```

// src/main.rs
mod state; mod errors; mod auth; mod models; mod routes;

use axum::{Router, routing::{get, post, put, delete}};
use sqlx::postgres::PgPoolOptions;
use tower_http::{trace::TraceLayer, cors::CorsLayer};
use state::AppState;

#[tokio::main]
async fn main() {
    dotenvy::dotenv().ok();
    tracing_subscriber::fmt::init();

    let db = PgPoolOptions::new()
        .max_connections(5)
        .connect(&std::env::var("DATABASE_URL").unwrap())
        .await
        .expect("failed to connect to db");

    sqlx::migrate!().run(&db).await.expect("migrations failed");

    let state = AppState {
        db,
        jwt_secret: std::env::var("JWT_SECRET").unwrap(),
    };

    let app = Router::new()
        .route("/health", get(|| async { "OK" })))
        .route("/auth/register", post(routes::auth::register))
        .route("/auth/login", post(routes::auth::login))
        .route("/tasks",
get(routes::tasks::list_tasks).post(routes::tasks::create_task))
        .route("/tasks/:id",
put(routes::tasks::update_task).delete(routes::tasks::delete_task))
        .layer(TraceLayer::new_for_http())

```



```

        .layer(CorsLayer::permissive())
        .with_state(state);

let listener = tokio::net::TcpListener::bind("0.0.0.0:3000").await.unwrap();
tracing::info!("TaskFlow listening on http://localhost:3000");
axum::serve(listener, app).await.unwrap();
}

```

19.9 Testing it end-to-end with curl

Terminal: `➤ a full register ➤ create ➤ list flow`

```

# Register
curl -X POST localhost:3000/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"email":"sid@example.com","password":"supersecret1"}'
# -> {"token":"eyJhbGciOi..." }

# Create a task (paste the token from above)
curl -X POST localhost:3000/tasks \
  -H 'Authorization: Bearer eyJhbGciOi...' \
  -H 'Content-Type: application/json' \
  -d '{"title":"Ship TaskFlow"}'

# List your tasks
curl localhost:3000/tasks -H 'Authorization: Bearer eyJhbGciOi...'

```

TIP

At this point TaskFlow is a genuinely production-shaped service: typed, validated, authenticated, logged, and backed by a real database — the same architecture pattern used by real Rust backend teams. Chapter 20 packages it for deployment.

■ Capstone checklist

Add `PUT /tasks/:id` to toggle `is_done`, following the same ownership-scoped pattern as `delete_task`.
 Add pagination to `list_tasks` using `LIMIT` / `OFFSET` query parameters.
 Write at least 3 integration tests in `tests/api.rs` covering register, login, and a protected route.

Deployment with Docker

Rust's biggest deployment advantage is that `cargo build --release` produces a single, small, self-contained binary with no runtime dependency (no Node.js, no JVM, no Python interpreter needed on the server). This makes Docker images for Rust backends extremely small and fast to start.

20.1 A multi-stage Dockerfile

Dockerfile — a two stage build keeps the final image small

```
# ---- build stage ----
FROM rust:1.79 AS builder
WORKDIR /app
COPY . .
RUN cargo build --release

# ---- runtime stage ----
FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y ca-certificates libssl3 && rm -rf
/var/lib/apt/lists/*
WORKDIR /app
COPY --from=builder /app/target/release/taskflow /app/taskflow
COPY migrations ./migrations
EXPOSE 3000
CMD ["/taskflow"]
```

NOTE

The **builder** stage has the full Rust toolchain (~1.5GB) but is discarded. Only the compiled binary (a few MB) is copied into the slim final image — a working TaskFlow image typically ends up under 100MB, versus several hundred MB for equivalent Node.js/Python images.

20.2 docker-compose for local development

docker-compose.yml — one command spins up both the API and Postgres

```
version: "3.9"
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: password
      POSTGRES_DB: taskflow
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]

  api:
    build: .
    depends_on: [db]
    environment:
      DATABASE_URL: postgres://postgres:password@db:5432/taskflow
      JWT_SECRET: dev-secret-change-me
```

```
ports: ["3000:3000"]

volumes:
  pgdata:
```

Terminal

```
docker compose up --build
```

20.3 Speeding up rebuilds with dependency caching

The naive Dockerfile above recompiles all dependencies on every code change. A common trick is to copy only Cargo.toml/Cargo.lock first, build a dummy project to cache dependencies, then copy the real source — so Docker's layer cache skips recompiling unchanged dependencies.

Dependency caching pattern — dramatically faster iterative builds

```
FROM rust:1.79 AS builder
WORKDIR /app
COPY Cargo.toml Cargo.lock ./
RUN mkdir src && echo 'fn main() {}' > src/main.rs && cargo build --release
COPY src ./src
RUN touch src/main.rs && cargo build --release
```

20.4 Where to deploy

Option	Good for
Fly.io / Railway	Fastest path from Dockerfile to a live URL, generous free tiers
AWS (ECS/Fargate or EC2)	Full control, integrates with RDS Postgres, scales to production traffic
Render	Simple git-push deploys with managed Postgres add-ons
A VPS + systemd	Full control, cheapest at small scale, more manual ops work

20.5 Running as a systemd service (bare VPS option)

systemd unit file — auto-restarts your binary on crash or reboot

```
# /etc/systemd/system/taskflow.service
[Unit]
Description=TaskFlow API
After=network.target

[Service]
ExecStart=/opt/taskflow/taskflow
EnvironmentFile=/opt/taskflow/.env
Restart=always
User=taskflow

[Install]
WantedBy=multi-user.target
```

Terminal

```
sudo systemctl daemon-reload
sudo systemctl enable taskflow
sudo systemctl start taskflow
sudo systemctl status taskflow
```

20.6 Environment & secrets checklist

- Never commit `.env` — add it to `.gitignore` immediately.
- Use a long, random `JWT_SECRET` in production — generate with `openssl rand -hex 32`.
- Run `sqlx migrate run` as a deploy step, not manually on the production database.
- Set `RUST_LOG=info` (or similar) in production so `tracing` output stays useful without being noisy.

■ Practice

Write a Dockerfile for TaskFlow and confirm `docker build .` succeeds locally. Bring up TaskFlow + Postgres together with `docker compose up` and run the curl flow from Chapter 19.

Best Practices & Where to Go Next

21.1 Idiomatic Rust checklist

- Run **cargo clippy** regularly — it catches non-idiomatic patterns and genuine bugs before review.
- Run **cargo fmt** on save so style is never a discussion point in code review.
- Prefer **&str**; function parameters over **&String**; — it's more flexible and idiomatic.
- Avoid **.clone()** as a first fix for borrow-checker errors — often a restructure (borrowing correctly, or using an index/id instead of a reference) is cleaner and faster.
- Keep functions small; let the type system (Result, Option, enums) express intent instead of comments.
- Write a few integration tests per route from day one — they catch regressions across chapters/refactors.

21.2 Common beginner mistakes

Mistake	Better approach
Fighting the borrow checker with <code>.clone()</code> everywhere	Learn to restructure ownership; clone sparingly and deliberately
Using <code>unwrap()</code> in request handlers	Use <code>?</code> with a central <code>AppError</code> (Chapter 17)
One giant <code>main.rs</code>	Split into modules early (Chapter 11) — <code>routes/models/db/errors</code>
Blocking calls inside <code>async fn</code> (e.g. <code>std::thread::sleep</code>)	Use <code>tokio::time::sleep</code> and <code>async-aware</code> libraries throughout
Ignoring compiler warnings	Treat every <code>`warning:`</code> as a to-do item — Rust's warnings are rarely noise

21.3 Where to go from here

- **gRPC backends** — explore the *tonic* crate for high-performance service-to-service APIs.
- **Message queues** — integrate Kafka or RabbitMQ using *rdkafka* or *lapin* for event-driven backend architectures.
- **Observability** — add OpenTelemetry tracing exporters on top of the ``tracing`` crate for production-grade monitoring.
- **WebAssembly** — the same ownership model that makes Rust great for backends also makes it a top choice for compiling to WASM.
- **Read real code** — the Axum, SQLx, and Tokio repositories are well-documented and a great way to see idiomatic patterns at scale.

21.4 A suggested 8-week practice plan

Week	Focus
1	Chapters 1-5 — syntax, ownership (do every exercise, no shortcuts)
2	Chapters 6-9 — structs, enums, pattern matching, error handling
3	Chapters 10-12 — traits, generics, modules, threads
4	Chapters 13-14 — async/Tokio, first Axum server
5	Chapter 15 — wire up Postgres with SQLx, run real migrations
6	Chapter 16-17 — auth, JWT, centralized error handling
7	Chapters 18-19 — testing, complete the TaskFlow capstone fully
8	Chapter 20 — Dockerize and deploy TaskFlow live; then extend it with one new feature of your own design (e.g. task comments, due-date reminders, or team sharing)

TIP

The single best way to solidify all of this is to keep extending TaskFlow after this course ends: add task categories, due dates, a search endpoint, rate limiting, or a second resource type entirely. Real fluency comes from the 5th and 6th time you write a handler, not the 1st.

APPENDIX

Cheat Sheet & Further Resources

A.1 Syntax quick reference

Concept	Syntax
Immutable / mutable variable	<code>let x = 5; / let mut x = 5;</code>
Function	<code>fn name(arg: Type) -> ReturnType { ... }</code>
Struct	<code>struct Name { field: Type }</code>
Enum	<code>enum Name { VariantA, VariantB(Type) }</code>
Trait	<code>trait Name { fn method(&self); }</code>
Generic function	<code>fn f(x: T) -> T</code>
Reference / mutable reference	<code>&value; / &mut; value</code>
Result	<code>Result → Ok(v) / Err(e)</code>
Option	<code>Option → Some(v) / None</code>
Error propagation	<code>some_fallible_call()?;</code>
Vec	<code>let v: Vec = vec![1, 2, 3];</code>
HashMap	<code>let m: HashMap = HashMap::new();</code>
Async function	<code>async fn f() { ... } / f().await;</code>
Spawn a task	<code>tokio::spawn(async { ... });</code>

A.2 Cargo command reference

Command	Purpose
<code>cargo new / cargo init</code>	Create a new project
<code>cargo build / --release</code>	Compile (debug / optimized)
<code>cargo run</code>	Build and run
<code>cargo check</code>	Fast type-check without a binary
<code>cargo add</code>	Add a dependency
<code>cargo test</code>	Run all tests
<code>cargo fmt</code>	Auto-format code

Command	Purpose
cargo clippy	Lint for idiomatic issues
cargo doc --open	Build and view documentation locally

A.3 The TaskFlow crate stack, at a glance

Layer	Crate(s)
Async runtime	tokio
Web framework	axum, tower-http
Serialization	serde, serde_json
Database	sqlx (postgres), uuid, chrono
Auth	jsonwebtoken, argon2
Errors	thiserror
Validation	validator
Logging	tracing, tracing-subscriber
Config	dotenvy

A.4 Further resources

- **The Rust Book** (doc.rust-lang.org/book) — the official, free, and excellent language reference.
- **Rust by Example** (doc.rust-lang.org/rust-by-example) — short, runnable snippets for every concept.
- **Axum documentation & examples** (github.com/tokio-rs/axum/tree/main/examples) — dozens of real handler patterns.
- **SQLx documentation** (github.com/launchbadge/sqlx) — query macros, migrations, connection pooling.
- **Zero To Production In Rust** (a well-known backend-focused Rust book) for going deeper on production concerns.
- **This Week in Rust** newsletter — a weekly digest to stay current with the ecosystem.

That's the full course — from `println!("Hello, backend world!")` in Chapter 2 to a deployed, authenticated, database-backed API in Chapter 20. The fastest way to keep the momentum going is to keep building on TaskFlow this week while it's fresh.

CHAPTER 22 · SPEED

Caching with Redis

Once TaskFlow has real traffic, some queries (like a public dashboard or a frequently-hit summary endpoint) benefit from caching instead of hitting Postgres every time. **Redis** is the standard in-memory store for this, and the *redis* crate gives you an async client that fits naturally next to Tokio and Axum.

22.1 Setup

Terminal

```
cargo add redis --features tokio-comp
```

Basic get/set with expiry using redis.rs

```
use redis::AsyncCommands;

async fn get_client() -> redis::Client {
    redis::Client::open("redis://127.0.0.1/").expect("invalid redis url")
}

async fn cache_task_count(client: &redis::Client, user_id: &str, count: i64) ->
redis::RedisResult<> {
    let mut conn = client.get_multiplexed_async_connection().await?;
    conn.set_ex(format!("task_count:{user_id}"), count, 60).await // expires in 60s
}

async fn get_cached_task_count(client: &redis::Client, user_id: &str) -> Option<i64> {
    let mut conn = client.get_multiplexed_async_connection().await.ok()?;
    conn.get(format!("task_count:{user_id}")).await.ok()
}
```

22.2 The cache-aside pattern

The most common caching strategy for a REST API: check the cache first; on a miss, query the database, then populate the cache for next time.

Cache-aside: check cache -> fallback to DB -> repopulate cache

```
async fn task_count(state: &AppState, user_id: &str) -> Result<i64, AppError> {
    if let Some(cached) = get_cached_task_count(&state.redis, user_id).await {
        return Ok(cached);
    }

    let count: i64 = sqlx::query_scalar("SELECT COUNT(*) FROM tasks WHERE user_id = $1")
        .bind(uuid::Uuid::parse_str(user_id).unwrap())
        .fetch_one(&state.db)
        .await?;

    let _ = cache_task_count(&state.redis, user_id, count).await;
    Ok(count)
}
```

TIP

Always set an expiry (TTL) on cached values in a task-management API — stale cached counts are confusing to users. A short TTL (30-120 seconds) is usually the right tradeoff between freshness and database load.

22.3 Cache invalidation

The classic hard problem: whenever a task is created, updated, or deleted, you must also invalidate (delete) the relevant cache key, or users will see outdated data.

Invalidate on every write that affects the cached value.

```
async fn invalidate_task_count(client: &redis::Client, user_id: &str) ->
redis::RedisResult<()> {
    let mut conn = client.get_multiplexed_async_connection().await?;
    conn.del(format!("task_count:{user_id}")).await
}

// call this at the end of create_task / delete_task handlers
```

■ Practice

Add Redis caching to the ``list_tasks`` endpoint with a 30-second TTL.

Invalidate the cache key inside ``create_task`` and ``delete_task`` so results never go stale for more than a few seconds.

Rate Limiting & Security

A backend exposed to the internet needs defenses beyond "does the code work": limiting abusive traffic, setting sane security headers, and following basic hygiene around secrets and input.

23.1 Rate limiting with tower-governor

Terminal

```
cargo add tower-governor
```

Limiting brute-force login attempts per IP

```
use tower_governor::{governor::GovernorConfigBuilder, GovernorLayer};

let governor_conf = GovernorConfigBuilder::default()
    .per_second(5) // 5 requests
    .burst_size(10) // allow short bursts up to 10
    .finish()
    .unwrap();

let app = Router::new()
    .route("/auth/login", post(routes::auth::login))
    .layer(GovernorLayer { config: Box::leak(Box::new(governor_conf)) });
```

23.2 Security headers with tower-http

Adding standard hardening headers

```
use tower_http::set_header::SetResponseHeaderLayer;
use axum::http::{HeaderName, HeaderValue};

let app = Router::new()
    .route("/health", get(|| async { "OK" })))
    .layer(SetResponseHeaderLayer::if_not_present(
        HeaderName::from_static("x-content-type-options"),
        HeaderValue::from_static("nosniff"),
    ));
```

23.3 Practical security checklist for a Rust API

- **Always validate and sanitize input** — use the `validator` crate (Chapter 17) on every request body.
- **Parameterize all SQL** — SQLx's `.bind()` already prevents SQL injection; never build raw SQL with string concatenation of user input.
- **Hash passwords with argon2** (Chapter 16), never store or log plain-text passwords.
- **Set short JWT expiries** (hours, not months) and support token refresh instead of long-lived tokens.
- **Enforce HTTPS in production** — terminate TLS at a reverse proxy (nginx, Caddy, or your cloud provider's load balancer) in front of the Rust service.

- **Restrict CORS** to known origins in production — `CorsLayer::permissive()` from Chapter 17 is for local development only.
- **Never log secrets** — scrub tokens/passwords from `tracing` output before it reaches log storage.

23.4 Dependency auditing

Terminal

```
cargo install cargo-audit
cargo audit
```

`cargo-audit` checks your `Cargo.lock` against the RustSec security advisory database — run it regularly (and in CI) to catch known-vulnerable dependencies before they ship.

■ Practice

Add rate limiting to `/auth/login` and `/auth/register` specifically — these are the most brute-forced endpoints on any API.

Run `cargo audit` on your TaskFlow project and resolve any flagged advisories.

Background Jobs & Scheduled Tasks

Not all backend work belongs inside an HTTP handler — sending a welcome email, generating a report, or cleaning up old data should happen in the background so requests stay fast.

24.1 Fire-and-forget with `tokio::spawn`

The request returns fast; the email sends independently.

```
async fn register_handler(/* ... */) -> Result<Json<AuthResponse>, AppError> {
    // ... create user, issue token ...

    let email = req.email.clone();
    tokio::spawn(async move {
        if let Err(e) = send_welcome_email(&email).await {
            tracing::error!(error = %e, "failed to send welcome email");
        }
    });

    Ok(Json(AuthResponse { token })) // responds immediately, doesn't wait for the email
}
```

NOTE

`tokio::spawn` is fine for lightweight, best-effort work. For anything that **must** eventually succeed (billing, notifications that can't be silently dropped if the server restarts), use a durable job queue instead — see 24.2.

24.2 Durable job queues

For jobs that must survive a server restart or be retried on failure, persist them and process from a queue. Two common approaches in the Rust ecosystem:

Approach	When to use it
Postgres-backed queue table + a worker loop	Simplest — no new infrastructure, good up to moderate volume
apalis / faktory / RabbitMQ + lapin	Higher throughput, retries, scheduling, and dashboards built in

A simple, effective durable job pattern for small to medium apps:

```
-- a minimal jobs table
CREATE TABLE jobs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    kind TEXT NOT NULL,
    payload JSONB NOT NULL,
    status TEXT NOT NULL DEFAULT 'pending', -- pending / running / done / failed
    attempts INT NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);
```

FOR UPDATE SKIP LOCKED lets multiple worker instances safely pull from the same queue.

```
async fn worker_loop(pool: PgPool) {
    loop {
        let job = sqlx::query_as::(<_, Job>{
            "UPDATE jobs SET status = 'running' WHERE id = (
                SELECT id FROM jobs WHERE status = 'pending' ORDER BY created_at LIMIT 1
            FOR UPDATE SKIP LOCKED
            ) RETURNING *"
        }).fetch_optional(&pool).await.unwrap();

        match job {
            Some(j) => process_job(&pool, j).await,
            None => tokio::time::sleep(std::time::Duration::from_secs(2)).await,
        }
    }
}
```

24.3 Scheduled tasks with tokio-cron-scheduler

Terminal

```
cargo add tokio-cron-scheduler
```

Cron-style scheduled maintenance jobs

```
use tokio_cron_scheduler::{JobScheduler, Job};

let sched = JobScheduler::new().await?;
sched.add(Job::new_async("0 0 3 * * **", |_uuid, _l| { // every day at 3 AM
    Box::pin(async move {
        tracing::info!("running nightly cleanup");
        // delete old completed tasks, expired tokens, etc.
    })
}))?.await?;
sched.start().await?;
```

■ Practice

Add a `jobs` table and a worker loop that processes a `send\_welcome\_email` job kind.
Add a nightly scheduled job that deletes completed tasks older than 90 days.

WebSockets with Axum

REST is great for request/response, but real-time features — live task updates across a team, notifications — need a persistent connection. Axum has WebSocket support built in.

25.1 A basic WebSocket handler

The simplest possible echo WebSocket

```
use axum::extract::ws::{WebSocket, WebSocketUpgrade, Message};
use axum::response::IntoResponse;

async fn ws_handler(ws: WebSocketUpgrade) -> impl IntoResponse {
    ws.on_upgrade(handle_socket)
}

async fn handle_socket(mut socket: WebSocket) {
    while let Some(Ok(msg)) = socket.recv().await {
        if let Message::Text(text) = msg {
            let reply = format!("echo: {text}");
            if socket.send(Message::Text(reply)).await.is_err() {
                break; // client disconnected
            }
        }
    }
}

// router: .route("/ws", get(ws_handler))
```

25.2 Broadcasting task updates to connected clients

For a feature like "see teammates' task updates live," use a Tokio broadcast channel: every connected client subscribes to it, and any handler that mutates a task publishes an event.

broadcastchannel — one sender, many WebSocket subscribers

```
use tokio::sync::broadcast;

#[derive(Clone)]
struct AppState {
    db: PgPool,
    task_events: broadcast::Sender<String>, // JSON-encoded events
}

async fn handle_socket(mut socket: WebSocket, state: AppState) {
    let mut rx = state.task_events.subscribe();
    while let Ok(event) = rx.recv().await {
        if socket.send(Message::Text(event)).await.is_err() {
            break;
        }
    }
}
```

```
// after successfully creating a task in create_task():  
let _ = state.task_events.send(serde_json::to_string(&task).unwrap());
```

TIP

This is conceptually similar to Socket.IO rooms in a Node.js backend, but implemented with Tokio's native broadcast channel — no extra library required for the basic pattern.

■ Practice

Add a `/ws` route to TaskFlow that broadcasts a message whenever any task is created. Test it by connecting two WebSocket clients (e.g. with `websocat` or a browser console) and confirming both receive the same event.

Performance, Profiling & Benchmarking

26.1 Benchmarking with criterion

Terminal

```
cargo add --dev criterion --features html_reports
```

benches/task_bench.rs — run with `cargo bench`

```
// benches/task_bench.rs
use criterion::{criterion_group, criterion_main, Criterion};

fn bench_title_validation(c: &mut Criterion) {
    c.bench_function("validate_title", |b| {
        b.iter(|| validate_title("a reasonably long task title for benchmarking"))
    });
}

criterion_group!(benches, bench_title_validation);
criterion_main!(benches);
```

26.2 Load testing your API

Benchmarks measure a function in isolation; load tests measure the whole server under realistic concurrent traffic. **k6** and **oha** are both good, simple options that work against any HTTP API regardless of language.

Terminal — measuring requests/sec and latency percentiles

```
# oha — quick, no-config load test
cargo install oha
oha -n 2000 -c 50 http://localhost:3000/health

# k6 — scriptable load tests
k6 run --vus 50 --duration 30s loadtest.js
```

26.3 Where Rust backends actually lose performance

- **Unnecessary .clone() calls** on large structs in hot paths — profile before assuming it matters, but it's the most common self-inflicted slowdown.
- **N+1 database queries** — the same problem as any ORM-based backend; batch queries or use joins instead of looping and querying per-row.
- **Blocking calls inside async handlers** — e.g. synchronous file I/O — stalls the whole Tokio worker thread; use `tokio::fs` or `spawn_blocking` for unavoidable blocking work.
- **Too few or too many database pool connections** — tune `max_connections` against your actual concurrent request volume and Postgres's own connection limit.

- **Debug builds in production** — always deploy with ``cargo build --release``; debug builds can be 10-100x slower.

26.4 Quick profiling with tracing spans

`#[instrument]` — near zero effort timing visibility per function

```
use tracing::instrument;

#[instrument(skip(pool))]
async fn list_tasks(pool: &PgPool, user_id: Uuid) -> Result<Vec<Task>, sqlx::Error> {
    // tracing automatically records how long this function took,
    // and nests it under the request span from TraceLayer
    sqlx::query_as("SELECT * FROM tasks WHERE user_id = $1")
        .bind(user_id)
        .fetch_all(pool)
        .await
}
```

■ Practice

Run ``oha`` against your local TaskFlow ``/health`` and ``/tasks`` endpoints and compare latency. Add ``#[instrument]`` to your three busiest handlers and observe the timing output in your logs.

Common Design Patterns in Rust Backends

Rust's type system encourages a slightly different set of patterns than object-oriented languages. These four show up constantly in real backend codebases, including larger versions of TaskFlow.

27.1 The Repository pattern

Instead of scattering `sqlx::query_as` calls across every handler, wrap database access behind a trait. This makes handlers easier to read and lets you swap the database layer out in tests.`

Handlers depend on the trait, not directly on SQLx — easy to mock in tests

```
use async_trait::async_trait;

#[async_trait]
trait TaskRepository {
    async fn list_for_user(&self, user_id: Uuid) -> Result<Vec<Task>, sqlx::Error>;
    async fn create(&self, user_id: Uuid, title: &str) -> Result<Task, sqlx::Error>;
}

struct PgTaskRepository { pool: PgPool }

#[async_trait]
impl TaskRepository for PgTaskRepository {
    async fn list_for_user(&self, user_id: Uuid) -> Result<Vec<Task>, sqlx::Error> {
        sqlx::query_as("SELECT * FROM tasks WHERE user_id = $1")
            .bind(user_id).fetch_all(&self.pool).await
    }
    async fn create(&self, user_id: Uuid, title: &str) -> Result<Task, sqlx::Error> {
        sqlx::query_as("INSERT INTO tasks (user_id, title) VALUES ($1, $2) RETURNING *")
            .bind(user_id).bind(title).fetch_one(&self.pool).await
    }
}
```

27.2 The Newtype pattern — type-safe IDs

A plain `Uuid` could accidentally be a user id where a task id was expected. Wrapping it in a tiny struct makes that a compile error instead of a runtime bug.

The newtype pattern — zero runtime cost, real compile-time safety

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, sqlx::Type)]
#[sqlx(transparent)]
struct TaskId(Uuid);

#[derive(Debug, Clone, Copy, PartialEq, Eq, sqlx::Type)]
#[sqlx(transparent)]
struct UserId(Uuid);

// fn assign(task: TaskId, user: UserId) — now swapping the arguments by mistake won't
// compile
```

27.3 The Builder pattern

Builder pattern — readable, optional configuration via chained calls

```
#[derive(Default)]
struct TaskFilterBuilder {
    is_done: Option<bool>,
    search: Option<String>,
    limit: Option<i64>,
}

impl TaskFilterBuilder {
    fn done(mut self, v: bool) -> Self { self.is_done = Some(v); self }
    fn search(mut self, s: &str) -> Self { self.search = Some(s.to_string()); self }
    fn limit(mut self, n: i64) -> Self { self.limit = Some(n); self }
}

// usage:
let filter = TaskFilterBuilder::default().done(false).limit(20);
```

27.4 Enums as type-safe state machines

We saw `TaskStatus` back in Chapter 6 — this is worth revisiting as a deliberate pattern. Modeling a workflow as an enum means invalid states (like "done" with no completion date) simply can't be represented.

Illegal states are unrepresentable — a hallmark of idiomatic Rust domain modeling

```
enum TaskState {
    Todo,
    InProgress { started_at: DateTime<Utc> },
    Done { started_at: DateTime<Utc>, completed_at: DateTime<Utc> },
}

impl TaskState {
    fn start(self) -> Self {
        match self {
            TaskState::Todo => TaskState::InProgress { started_at: Utc::now() },
            other => other, // already started/done — no-op
        }
    }
}
```

■ Practice

Wrap your `TaskFlow` SQLx queries behind a `TaskRepository` trait and re-point your handlers at it. Convert your `Task`'s `is_done: bool` field into a `TaskState` enum and update every place that reads it.

Framework & Ecosystem Comparison

This course uses Axum + SQLx + Postgres because it's currently the most common combination in new Rust backend projects, but it's worth knowing the alternatives and why you might pick them.

28.1 Web frameworks

Framework	Notes
Axum	Built by the Tokio team, tightly integrated with Tokio/tower. Composable, minimal magic, currently the most popular choice for new projects. Used throughout this course.
Actix-web	Mature, extremely high raw throughput benchmarks, actor-model heritage. Slightly more idiosyncratic API than Axum, still widely used in production.
Rocket	Emphasizes developer ergonomics and macros for a very declarative feel, closer to frameworks like Flask/Rails in spirit. Historically slower to reach stable async support.
warp	Filter-based, functional composition style. Powerful but has a steeper learning curve for beginners than Axum.

28.2 Database access: SQLx vs Diesel

Tool	Notes
SQLx	Async-first, you write real SQL, compile-time query checking against a live database. Used throughout this course — fits naturally with Axum + Tokio.
Diesel	A full ORM with a query-builder DSL and its own migration system, historically synchronous (async support has matured but is less central to its design). Strong choice if you want more abstraction over raw SQL.
SeaORM	Async ORM built on SQLx, closer to an ActiveRecord-style API if you're coming from Rails/Django and want that pattern in Rust.

28.3 Database engines

- **PostgreSQL** — the default recommendation for most new backends: rich feature set (JSONB, full-text search, extensions), excellent SQLx support, used throughout this course.
- **SQLite** — excellent for local development, tests, embedded/single-binary deployments, or genuinely small-scale apps; SQLx supports it with the same API you already know.
- **MySQL** — also well supported by SQLx; a reasonable choice mainly if your infrastructure or team already standardizes on it.

28.4 A simple decision guide

If you want...	Choose
The most common, best-documented, beginner-friendly path	Axum + SQLx + Postgres (this course)
Maximum raw request throughput benchmarks	Actix-web
Rails/Django-like ORM ergonomics	SeaORM or Diesel
Zero external database service for local dev/tests	SQLite via SQLx

NOTE

There is no universally "correct" stack — the Axum/SQLx/Postgres combination in this course is a strong, well-documented default for a first production Rust backend, not the only valid choice.

CI/CD with GitHub Actions

A CI pipeline catches broken builds, failing tests, and lint issues before they reach production — and for a compiled language like Rust, it also means teammates never pull code that doesn't build.

29.1 A CI workflow: build, test, lint

.github/workflows/ci.yml — runs on every push and pull request

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: password
          POSTGRES_DB: taskflow_test
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready --health-interval 10s --health-timeout 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
      - uses: Swatinem/rust-cache@v2 # caches target/ between runs for faster CI
      - name: Check formatting
        run: cargo fmt --check
      - name: Lint
        run: cargo clippy -- -D warnings
      - name: Run migrations
        run: |
          cargo install sqlx-cli --no-default-features --features postgres
          sqlx migrate run
      - name: Run tests
        run: cargo test
        env:
          DATABASE_URL: postgres://postgres:password@localhost:5432/taskflow_test
```

29.2 A deploy workflow

Deploy only after CI passes on main — build once, deploy the exact tested image

```
# .github/workflows/deploy.yml
name: Deploy
on:
```

```

push:
  branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    needs: [] # in practice, make this depend on the CI job passing first
    steps:
      - uses: actions/checkout@v4
      - name: Build and push Docker image
        run: |
          docker build -t registry.example.com/taskflow:${{ github.sha }} .
          docker push registry.example.com/taskflow:${{ github.sha }}
      - name: Deploy
        run: flyctl deploy --image registry.example.com/taskflow:${{ github.sha }}
    env:
      FLY_API_TOKEN: ${ secrets.FLY_API_TOKEN }

```

TIP

Keep ``cargo fmt --check`` and ``cargo clippy -- -D warnings`` in every CI pipeline from day one on a new project — retrofitting formatting/lint rules onto an existing large codebase later is far more painful than starting clean.

■ Practice

Add a GitHub Actions CI workflow to your TaskFlow repository that runs `fmt`, `clippy`, and `cargo test` against a real Postgres service container.

Confirm the workflow fails correctly by intentionally introducing a `clippy` warning, then fix it.

A Beginner's Field Guide to Compiler Errors

New Rust developers often say "if it compiles, it works" — but getting there means learning to read error messages, which are unusually detailed and, once you know the vocabulary, genuinely helpful.

30.1 "cannot move out of ... because it is borrowed"

Trigger

```
fn main() {
    let s = String::from("hello");
    let r = &s;
    let s2 = s; // error: cannot move `s` because it is borrowed here
    println!("{r}");
}
```

What it means: you tried to move ownership of `s` into `s2` while `r` still needed to read it. **Fix:** either clone the value (`let s2 = s.clone();`) if you truly need two independent copies, or restructure so the move happens after the last use of the reference.

30.2 "cannot borrow ... as mutable more than once"

Trigger

```
let mut v = vec![1, 2, 3];
let a = &mut v;
let b = &mut v; // error: second mutable borrow while `a` is still alive
```

What it means: you tried to have two mutable references to the same data at once. **Fix:** finish using `a` before creating `b` (often just reordering code), or restructure to avoid needing two simultaneous mutable views — e.g. by working with indices instead of references.

30.3 "the trait bound ... is not satisfied"

Trigger

```
#[derive(Debug)]
struct Task { id: u32 }

fn main() {
    let t1 = Task { id: 1 };
    let t2 = Task { id: 1 };
    println!("{}", t1 == t2); // error: binary operation `==` cannot be applied to
    `Task`
}
```

What it means: `==` requires the `PartialEq` trait, which `Task` doesn't implement. **Fix:** add `#[derive(PartialEq)]` above the struct (alongside `Debug`).

30.4 "expected struct X, found struct Y" (mismatched types)

Usually exactly what it says — you passed the wrong type to a function, or forgot to `.await`` an async call (leaving you with a `Future` instead of a `T`), or forgot a `?`.unwrap()`` on a `Result` when a plain `T` was expected. Read the *expected* vs *found* types carefully — the fix is almost always visible right there.

30.5 General strategy when stuck

- 1 Read the **first** error only — later errors are often just consequences of the first one.
- 2 Look for the **help:** line — the compiler frequently suggests the exact fix (add ``&``, add `.clone()``, add a `derive`, etc.).
- 3 Run **cargo check** instead of `cargo build`` while iterating — it's faster and gives the same errors.
- 4 If a lifetime or generic error feels opaque, try writing a smaller, standalone reproduction of just that function — it's often much clearer in isolation.
- 5 Search the exact error code (e.g. `E0502``) — `rustc --explain E0502`` prints a detailed explanation with examples, directly from the compiler.

Terminal — get a full explanation for any error code

```
rustc --explain E0502
```

■ Practice

Intentionally introduce a borrow-checker error into a small function, read the compiler's suggestion, and fix it without looking anything up.

Run `rustc --explain`` on two or three error codes you've hit while working through this course.

gRPC & GraphQL in Rust

REST covers the large majority of backend APIs, including everything built in this course. But it's worth knowing when — and how — Rust supports the two other common API styles you'll encounter in real systems.

31.1 gRPC with tonic

gRPC is a binary, contract-first RPC protocol built on HTTP/2 and Protocol Buffers. It shines for **service-to-service** communication inside a backend (microservices talking to each other), where strict typed contracts and low latency matter more than human-readable payloads.

proto/tasks.proto — the contract shared across any language's client/server

```
// proto/tasks.proto
syntax = "proto3";
package tasks;

service TaskService {
  rpc GetTask (GetTaskRequest) returns (Task);
}

message GetTaskRequest { string id = 1; }
message Task {
  string id = 1;
  string title = 2;
  bool is_done = 3;
}
```

Terminal — tonic-build generates Rust types & traits from the .proto file at compile time

```
cargo add tonic prost
cargo add --build tonic-build
```

A generated trait — implement it, and tonic handles serialization, HTTP/2 framing, etc

```
#[tonic::async_trait]
impl TaskService for MyTaskService {
  async fn get_task(
    &self,
    request: tonic::Request<GetTaskRequest>,
  ) -> Result<tonic::Response<Task>, tonic::Status> {
    let id = request.into_inner().id;
    // fetch from database...
    Ok(tonic::Response::new(Task { id, title: "Example".into(), is_done: false }))
  }
}
```

31.2 GraphQL with async-graphql

GraphQL lets clients request exactly the fields they need in a single round trip — useful for frontend-heavy applications with varied, evolving data needs (a mobile app and a web dashboard pulling different shapes of the same underlying data).

Terminal

```
cargo add async-graphql async-graphql-axum
```

A minimal GraphQL query root, served through the same Axum Router as your REST endpoints

```
use async_graphql::{Object, Schema, EmptyMutation, EmptySubscription};

struct QueryRoot;

#[Object]
impl QueryRoot {
    async fn task(&self, id: String) -> Task {
        Task { id, title: "Example".into(), is_done: false }
    }
}

type TaskSchema = Schema<QueryRoot, EmptyMutation, EmptySubscription>;

// mount at /graphql alongside your existing Axum routes with
async_graphql_axum::GraphQL
```

31.3 Choosing between REST, gRPC, and GraphQL

Style	Best fit
REST (this course)	Public APIs, simple CRUD, maximum tooling/client compatibility, human-debuggable payloads
gRPC	Internal service-to-service calls, strict typed contracts, streaming, performance-critical paths
GraphQL	Client-driven data shapes, multiple frontends with different needs, avoiding over/under-fetching

TIP

Most real systems mix styles — a REST or GraphQL edge API for clients, with gRPC connecting internal services behind it. Nothing about Axum, Tokio, or SQLx changes if you add gRPC or GraphQL alongside your existing REST routes — everything from Chapters 13-19 still applies underneath.

Structuring Larger Rust Backends

TaskFlow's routes/models/db split (Chapter 11 & 19) works well up to a moderate size. As a backend grows past a handful of resources, a clearer layering discipline keeps things maintainable.

32.1 A layered architecture

Layer	Responsibility
HTTP layer (routes/handlers)	Parse requests, call the service layer, shape HTTP responses. No business logic or raw SQL here.
Service layer	Business rules — e.g. "a task can't be marked done twice", "a free-tier user can have at most 50 tasks". Depends on repositories, not on Axum.
Repository layer	Database access only (Chapter 27's Repository pattern). No business rules.
Domain models	Plain structs/enums representing your core concepts, independent of both HTTP and database representations.

Business rules live in the service layer, fully independent of Axum and SQLx.

```
// service layer – depends on the repository trait, not a concrete database
struct TaskService<R: TaskRepository> {
    repo: R,
}

impl<R: TaskRepository> TaskService<R> {
    async fn create_task(&self, user_id: Uuid, title: &str) -> Result<Task, AppError> {
        if title.trim().is_empty() {
            return Err(AppError::Validation("title cannot be empty".into()));
        }
        let existing_count = self.repo.count_for_user(user_id).await?;
        if existing_count >= 50 {
            return Err(AppError::Validation("task limit reached".into()));
        }
        Ok(self.repo.create(user_id, title).await?)
    }
}

// HTTP layer – thin, just glues the request to the service
async fn create_task_handler(
    State(state): State<AppState>,
    user: AuthUser,
    Json(req): Json<CreateTask>,
) -> Result<Json<Task>, AppError> {
    let task = state.task_service.create_task(user.id(), &req.title).await?;
    Ok(Json(task))
}
```

TIP

This separation is exactly what makes the service layer unit-testable without a real database — swap in a fake `TaskRepository`` implementation (an in-memory `Vec`) in tests, and every business rule test runs in microseconds instead of requiring Postgres.

32.2 Workspaces for multi-crate projects

Once a backend spans multiple deployables (an API server, a background worker, a CLI admin tool) that share domain logic, split them into a Cargo workspace (introduced in Chapter 11) so the shared code lives in one place.

One shared core crate, two binaries built on top of it

```
taskflow-workspace/  
■■■ Cargo.toml # [workspace] members = ["core", "api", "worker"]  
■■■ core/ # domain models, business logic, repository traits  
■■■ api/ # the Axum HTTP server (depends on core)  
■■■ worker/ # the background job processor (also depends on core)
```

32.3 Feature flags for optional functionality

Compile email support in or out depending on the deployment target

```
# Cargo.toml  
[features]  
default = []  
email = ["dep:lettre"]  
  
[dependencies]  
lettre = { version = "0.11", optional = true }
```

32.4 A pragmatic growth path

- 1 Start flat, like TaskFlow in Chapter 19 — routes/models/db in one crate. This is the right call for small projects and MVPs; don't over-engineer early.
- 2 Introduce the Repository pattern (Chapter 27) once you have more than a couple of resources, so handlers stop directly depending on SQLx.
- 3 Add an explicit service layer once business rules start appearing inside handlers — that's the signal it's time.
- 4 Split into a Cargo workspace only once you actually have a second binary (a worker, a CLI) that needs to share domain code — don't split preemptively.

Rust Backend Interview Questions & Answers

A quick-reference set of the questions that come up most often in Rust backend interviews, with short model answers you can expand on in your own words.

What is ownership, and why does Rust need it?

Ownership is Rust's compile-time system for tracking which variable is responsible for freeing a value's memory. Every value has exactly one owner; when the owner goes out of scope, the value is dropped. It replaces both manual memory management (like C) and garbage collection (like Java/Go) with zero runtime cost, while still preventing use-after-free and double-free bugs.

What's the difference between `String` and `&str`?

`String` is an owned, growable, heap-allocated string. `&str` is a borrowed view (a pointer + length) into string data owned elsewhere — either a `String` or a string literal. Function parameters should usually accept `&str` for flexibility; return `String` when you need to hand back ownership of new data.

Explain the borrow checker's core rule.

At any point, a value can have either exactly one mutable reference, or any number of immutable references — never both at once. This single rule is what prevents data races at compile time.

What is the difference between `Vec` and `&[T]`?

`Vec` is an owned, growable, heap-allocated array. `&[T]` (a slice) is a borrowed, fixed-size view into a contiguous sequence of `T` — it could point into a `Vec`, an array, or another slice. Prefer `&[T]` as a function parameter when you only need to read the data.

How does Rust handle null?

Rust has no null. Values that might be absent are wrapped in `Option` (`Some(value)` or `None`). The compiler forces you to handle both cases before accessing the inner value, eliminating null-pointer exceptions as a class of bug.

What's the difference between `Result` and `Option`?

`Option` represents presence or absence of a value with no further detail. `Result` represents success (`Ok(T)`) or failure with an explicit error value (`Err(E)`) — use it whenever the caller needs to know *why* something failed, not just that it did.

What does the `?` operator do?

Inside a function returning `Result` or `Option`, `?` unwraps an `Ok/Some` value and continues execution, or returns early with the `Err/None`, propagating it to the caller. It also converts compatible error types automatically via the `From` trait, which is why apps often centralize error handling around one `AppError` enum.

What is a trait, and how does it compare to an interface?

A trait defines a set of methods a type must implement, similar to an interface in Java/TypeScript. Unlike most interfaces, traits can supply default method implementations, and Rust supports both static dispatch (`impl Trait` / generics, resolved at compile time) and dynamic dispatch (Box, resolved at runtime).

What is the difference between async concurrency and OS threads?

OS threads are managed by the operating system and are relatively heavyweight (each has its own stack, typically megabytes). Async tasks are lightweight, cooperatively-scheduled units run by a runtime like Tokio on a small pool of OS threads — ideal for I/O-bound work like handling thousands of concurrent HTTP connections, since tasks yield control while waiting instead of blocking a thread.

Why does Rust require an external crate like Tokio for async instead of a built-in runtime?

Different applications have very different needs (a CLI tool, an embedded device, and a web server all want different scheduling tradeoffs), so Rust's standard library only defines the async building blocks (the `Future` trait, `async/await` syntax) and leaves the actual runtime as a pluggable choice. Tokio is the de-facto standard for backend/networked applications.

How would you structure error handling across a whole Axum API?

Define one `AppError` enum covering every failure mode (not found, unauthorized, validation, database, etc.), implement Axum's `IntoResponse` for it so it converts into a consistent JSON error response, and have every handler return `Result` so the `?` operator can propagate errors from database calls, JSON parsing, and validation with minimal boilerplate.

How do you prevent SQL injection in a Rust backend?

Use parameterized queries via SQLx's `.bind()` (or an equivalent in Diesel), never format user input directly into a SQL string. SQLx additionally offers compile-time query checking against your real schema, catching many mistakes before the code even runs.

What's the difference between `Arc` and just `Mutex`?

`Mutex` provides safe, exclusive mutable access to `T`, but by itself it's only valid within a single owner. `Arc` (Atomic Reference Count) allows multiple owners across threads by tracking a shared reference count. Combining them — `Arc` — lets multiple threads or async tasks safely share and mutate the same data.

Why might you choose Rust over Node.js or Python for a backend service?

Primarily for CPU-bound throughput, memory efficiency, and eliminating whole classes of runtime bugs (null errors, data races) at compile time — valuable at scale or for latency-sensitive services. The tradeoff is a steeper learning curve and typically slower initial development speed for CRUD-heavy, less performance-critical services, where Node.js/Python's larger ecosystems can move faster.

What is the difference between `.unwrap()` and the `?` operator?

`.unwrap()` panics immediately if the value is `Err/None`, crashing the current thread. The `?` operator instead returns early from the current function with the error, letting the caller decide what to do — appropriate for library and handler code, whereas `unwrap()` is generally reserved for tests, prototypes, or cases you've proven can never actually fail.

What does Send and Sync mean?

Send means a type can be safely transferred to another thread. Sync means a type can be safely shared by reference (&T;) across threads. Most types are both automatically; the compiler enforces these as trait bounds on anything crossing a thread boundary (e.g. inside `tokio::spawn`), which is part of how Rust guarantees thread safety at compile time.

What's the difference between static dispatch and dynamic dispatch in Rust?

Static dispatch (generics / impl Trait) resolves which concrete implementation to call at compile time, producing specialized, faster code but a larger binary (monomorphization). Dynamic dispatch (Box) resolves the implementation at runtime via a vtable, trading a small performance cost for the ability to store different concrete types behind one common interface.

How do you handle database connection pooling in an async Rust API?

Create one pool (e.g. `sqlx::PgPool`) at startup with a bounded `max_connections`, wrap it in your shared `AppState`, and clone the pool handle (cheap — it's an `Arc` internally) into each request via Axum's `State` extractor. Every handler borrows a connection from the pool for the duration of its query and returns it automatically when the future completes.

What's the purpose of the `sqlx::migrate!` macro?

It embeds your SQL migration files into the compiled binary at build time and, when run against a pool at startup, applies any migrations not yet recorded in a tracking table — so deploying the binary is enough to also bring the schema up to date, without a separate manual migration step.

How would you version a public REST API?

Common approaches include a URL prefix (`/v1/tasks`, `/v2/tasks`), a custom `Accept` header, or a query parameter. URL-prefix versioning is the most common and easiest for clients to reason about; the key discipline is keeping v1 handlers frozen once v2 exists, rather than silently changing behavior under an existing version.

What is backpressure, and why does it matter for a Rust backend?

Backpressure is a mechanism for signaling upstream producers to slow down when a consumer can't keep up — for example, a bounded channel (`tokio::sync::mpsc::channel(n)`) that blocks senders once full. Without it, a slow downstream dependency (like a struggling database) can cause unbounded memory growth as requests queue up faster than they're processed.

How do you keep secrets (DB passwords, JWT secret) out of source control?

Load them from environment variables at runtime (via `dotenv` locally, and real environment variables or a secrets manager in production), keep a committed `.env.example` with placeholder keys for documentation, and add the real `.env` to `.gitignore`. Never hardcode secrets into source files.

Glossary of Terms

Term	Definition
Ownership	Rust's system where each value has exactly one variable responsible for freeing it.
Borrowing	Temporarily accessing a value via a reference (&T; or &mut; T) without taking ownership.
Lifetime	A compile-time annotation describing how long a reference remains valid.
Crate	A compilation unit / package in Rust — either a binary (executable) or a library.
Cargo	Rust's official build tool and package manager.
Trait	A definition of shared behavior (methods) that types can implement, similar to an interface.
Generic	Code written to work with any type satisfying given trait bounds, resolved at compile time.
Struct	A custom data type that groups related named fields together.
Enum	A type representing one of several possible variants, each optionally carrying its own data.
Option	Rust's built-in type for an optional value: Some(T) or None. Replaces null.
Result	Rust's built-in type for a fallible operation: Ok(T) on success or Err(E) on failure.
Panic	An unrecoverable runtime error that unwinds (or aborts) the current thread.
Future	A value representing an asynchronous computation that hasn't completed yet.
async/await	Syntax for writing non-blocking code that reads like synchronous code.
Tokio	The most widely used async runtime for Rust, especially for networked/backend applications.
Axum	A web application framework built on Tokio, used to build TaskFlow throughout this course.
SQLx	An async SQL toolkit with compile-time query checking, used for TaskFlow's Postgres access.
Middleware	Code that runs before/after a request handler — logging, auth checks, CORS, etc.
Extractor	An Axum concept: a function parameter type that pulls data out of an incoming request (e.g. Json, Path, State).

Term	Definition
JWT	JSON Web Token — a signed, self-contained token used to represent an authenticated user.
Migration	A versioned SQL script that evolves a database schema over time.
Race condition	A bug where the correctness of a program depends on the unpredictable timing of concurrent operations — prevented at compile time in safe Rust.
Idiomatic	Code written the way experienced Rust developers conventionally write it, as opposed to code that merely compiles.

End of course. TaskFlow — and everything you now know to extend it — is yours to keep building on.

APPENDIX D

20 Practice Projects to Build After This Course

The fastest way to move from "followed a tutorial" to "comfortable Rust backend developer" is building a handful of these on your own, without a course to follow. Each includes the main new concept it forces you to practice beyond TaskFlow.

Project	What it teaches
URL shortener	Generate short codes, redirect, track click counts. Practice: unique constraints, redirects with Axum, simple analytics counters.
Personal finance tracker API	Track income/expenses by category with monthly summaries. Practice: aggregation queries (SUM, GROUP BY), date-range filtering.
Multi-tenant note-taking API	Notes with folders, shared between team members. Practice: role-based permissions, many-to-many relations.
Real-time chat backend	Rooms, message history, typing indicators. Practice: WebSockets (Chapter 25) at real scale, message persistence.
File upload & storage service	Accept file uploads, store in S3-compatible storage, return signed URLs. Practice: multipart form handling, streaming large bodies.
Rate-limited public API with API keys	Issue API keys instead of user logins, enforce per-key quotas. Practice: custom extractors, Redis-backed rate limiting.
Job board API	Postings, applications, search/filter by tags and location. Practice: full-text search with Postgres, complex filtering.
Webhook delivery system	Accept webhook subscriptions, deliver events with retries on failure. Practice: background job queues (Chapter 24), exponential backoff.
Polling/voting API	Create polls, vote once per user, live result updates. Practice: WebSocket broadcast (Chapter 25), preventing duplicate votes with unique constraints.

Project	What it teaches
E-commerce inventory service	Products, stock levels, reservations during checkout. Practice: database transactions, optimistic/pessimistic locking for stock.
Habit tracker with streak calculation	Daily check-ins, streak and completion-rate calculation. Practice: date/time logic with chrono, scheduled jobs for streak resets.
URL health monitor	Periodically pings a list of URLs, alerts on downtime. Practice: tokio-cron-scheduler (Chapter 24), external HTTP calls with reqwest.
Recipe API with ingredient scaling	Recipes with servings-based scaling (echoes the recipe widget in this course's toolset). Practice: numeric transformation logic, nested JSON structures.
Feature flag service	Toggle features per user/environment without redeploying. Practice: caching (Chapter 22), low-latency read-heavy design.
Event ticketing API	Events, seat/ticket reservations with limited inventory. Practice: preventing overselling under concurrent requests — a great real test of transactions.
Simple blog/CMS API	Posts, tags, drafts vs published, pagination. Practice: consolidating everything from Chapters 14-19 into a second, independent project.
Team task board (Kanban)	Extend TaskFlow itself: boards, columns, drag-and-drop ordering. Practice: ordering/position logic, team-based authorization instead of pure ownership.
Currency exchange rate cache	Fetch rates from a public API, cache and serve them. Practice: outbound HTTP calls with reqwest, cache-aside pattern (Chapter 22) end to end.
Log aggregation endpoint	Accept structured logs from other services, store and query them. Practice: high write-throughput design, time-series-friendly schema.
Multiplayer game score API	Submit scores, leaderboards, anti-cheat sanity checks. Practice: ranking queries (window functions), input validation under adversarial input.

TIP

Pick projects that overlap with things you already care about — given your interest in agri-tech and the family Ayurvedic practice, a farm input tracker or clinic appointment API (built the TaskFlow way: Axum + SQLx + JWT) would double as both practice and a genuinely useful tool.

HTTP Status Code Quick Reference

The status codes you'll actually use across TaskFlow-style APIs, mapped to when to return each one.

Code	When to use it
200 OK	Successful GET, PUT, or a general successful request with a body to return.
201 Created	Successful POST that created a new resource — return it in the body, often with a Location header.
204 No Content	Successful request with nothing to return — a common choice for DELETE.
400 Bad Request	Malformed input — failed validation, invalid JSON shape, bad query parameters.
401 Unauthorized	Missing or invalid authentication — no valid JWT, or login credentials wrong.
403 Forbidden	Authenticated, but not allowed to perform this action (e.g. accessing another user's private task).
404 Not Found	The resource (or route) doesn't exist — a missing task id, an unknown endpoint.
409 Conflict	The request conflicts with current state — e.g. a duplicate email on registration.
422 Unprocessable Entity	Well-formed request, but semantically invalid — some APIs use this instead of 400 for validation failures specifically.
429 Too Many Requests	Rate limit exceeded (Chapter 23) — include a Retry-After header if possible.
500 Internal Server Error	An unexpected failure on the server side — database down, a bug. Never leak internal details in the response body (Chapter 17's AppError pattern handles this).
503 Service Unavailable	The server is temporarily unable to handle the request — during maintenance, or when a critical dependency (like the database) is unreachable.